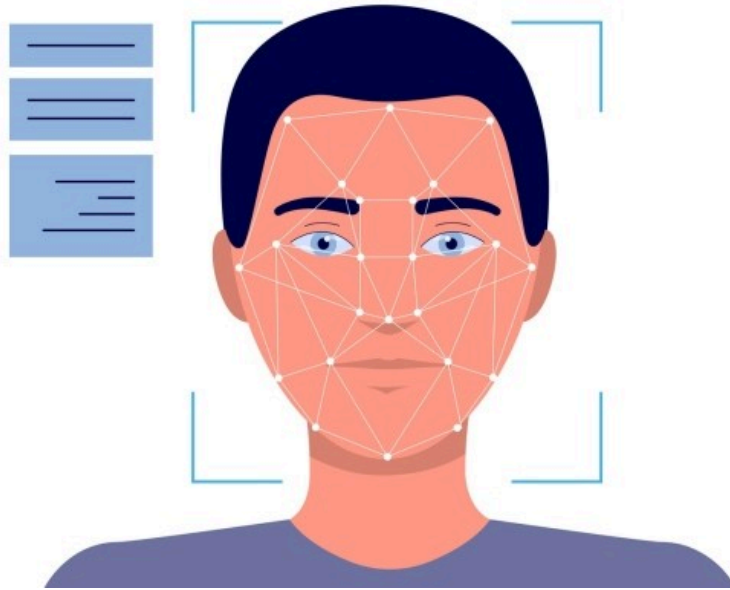


Hands-on Project: Facial Recognition System with Siamese Networks



1. Problem Statement

The task of this project is to develop a Siamese Network that can accurately identify whether a given image contains the face of celebrity or not. The network should be trained and evaluated on a separate test set, and should achieve good performance on the task. The network should also be able to generalize to unseen images of celebrities, and should be able to handle variations in pose, lighting, and background. The final model should be able to classify an image as containing a celebrity's face with high confidence.

2. Required Setup for the Project

2.1 Installing Dependencies

These modules are required if you are trying to run the Colab notebook in the local Jupyter environment, it is not required to install them on Google Colab as they are pre-installed in Colab already.

```
In [2]: # !pip install tensorflow tensorflow-gpu opencv-python matplotlib
```

2.2 Importing the required libraries (10 Marks)

```
In [1]: # Importing necessary libraries for plotting, handling arrays and directories, and

# Import libraries for file and directory manipulation
import os

# Import numpy for array manipulation and numerical computing
import numpy as np

# import cv2 for image manipulation
import cv2

# import random to generate random numbers
import random

# matplotlib for plotting plots
import matplotlib.pyplot as plt

# import tensorflow and dependencies for CNNs
import tensorflow as tf
from tensorflow.keras import layers, losses, optimizers, metrics
from tensorflow.keras.models import Sequential, Model

# import the convolutional 2D layer, Input layer and the dropout layer by completing
from tensorflow.keras.layers import Conv2D, Dense, Input, Flatten, MaxPooling2D, Dr

# import warnings to suppress unnecessary warning messages
import warnings
warnings.filterwarnings("ignore")

# import cv2_imshow for visualizing images
from google.colab.patches import import cv2_imshow
```

2.3 Limiting the usage of GPU

Using the complete GPU available might lead to out of memory error, so we will need to limit the usage of GPU memory.

```
In [2]: gpus = tf.config.experimental.list_physical_devices('GPU')

for gpu in gpus:
    tf.config.experimental.set_memory_growth(gpu, True)
```

- The `list_physical_devices()` function returns a list of physical devices of the specified device type, which in this case is 'GPU'. The `set_memory_growth()` function enables memory growth for the specified device, which allows the GPU to dynamically allocate memory as needed rather than allocating all available memory upfront. This can be useful when training large models that may not fit in the GPU memory.
- By enabling memory growth, the GPU will allocate only the necessary memory for the current batch of data, and will release the memory when it is no longer needed. This can

help prevent Out of Memory (OOM) errors and improve the efficiency of the training process.

- This above code is typically used when training deep learning models with large datasets on a GPU, as it allows the GPU to allocate memory more efficiently and prevent OOM errors.

2.4 Loading and Preprocessing Data (25 Marks)

```
In [3]: from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

2.4.1 Unzipping the content of the dataset provided (5 Marks)

```
In [4]: # Please input the path of the file by removing the blank spaces
!unzip -q "/content/drive/MyDrive/CelebrityFacesDataset-1.zip"
```

```
In [5]: class PreProcessing:

    # Loading the train and validation images from their path
    def __init__(self, verbose=True):
        """
        Initialize the PreProcessing class.

        Parameters:
            verbose (bool): Whether to print out information during the processing.
                           Defaults to True.
        """
        self.train_images = []
        self.val_images = []
        self.verbose = verbose
        self.train_path = "/content/CelebrityFacesDataset/CelebrityFacesDatasetTrain"
        self.val_path = "/content/CelebrityFacesDataset/CelebrityFacesDatasetTest"
        self.train_images = self.load_image(self.train_path)
        self.val_images = self.load_image(self.val_path)

    # Defining load_image function to load images
    def load_image(self, path, verbose=False):
        """
        This function loads images from a given directory, converts them to grayscale.
        :param path: The directory where the images are located.
        :param verbose: Whether to print the progress of the function.
        :return: A numpy array of the loaded, processed images.
        """
        images = []
        for folder in os.listdir(path):
            folder_path = os.path.join(path, folder)
            for filename in os.listdir(folder_path):
                file_path = os.path.join(folder_path, filename)
```

```

        img = cv2.imread(file_path, cv2.IMREAD_UNCHANGED)
        # img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
        img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
        img = cv2.resize(img, (105, 105))
        img = np.reshape(img, (105, 105, 1))
        images.append(img)
    return np.stack(images, axis=0)

def get_triplet(self, shape):
    """
    Return indices of anchor, positive and negative images from the same class

    Parameters:
    shape (tuple): shape of the data

    Returns:
    tuple: Indices of anchor, positive and negative images
    """
    class_1 = random.randint(0, shape[0] - 1)
    class_2 = random.randint(0, shape[0] - 1)
    while class_1 == class_2:
        class_2 = random.randint(0, shape[0] - 1)
    a = (class_1)
    p = (class_1)
    n = (class_2)
    return a, p, n

def get_triplet_batch(self, batch_size, train_data=True):
    """
    This function generates a batch of triplets, where the first two images are

    Parameters:
        batch_size (int): The number of triplets to generate.
        train_data (bool, optional): Whether to use the training data or test d

    Returns:
        tuple: A tuple containing three numpy arrays of shape (batch_size, 105,
    """
    anchors = []
    positives = []
    negatives = []
    if train_data:
        X = self.train_images
    else:
        X = self.val_images
    for i in range(batch_size):
        anchor, positive, negative = self.get_triplet(X.shape)
        anchors.append(X[anchor])
        positives.append(X[positive])
        negatives.append(X[negative])
    return np.array(anchors), np.array(positives), np.array(negatives)

```

2.4.2 Explain the following functions that are used in the code above

1. **load_image()** function (5 Marks) - Write your answer here:

The `load_image()` function is a helper function that goes into a folder that is full of pictures and prepares those pictures so a machine-learning algorithm can understand them. It reads each picture, cleans it up, makes all pictures the same size, converts them to black-and-white, and then returns them in a neat, organized format.

Step by Step Explanation

1. The function receives three inputs. The first one is automatically passed (self) and simply refers to the object itself. The second input is the folder path from where the images should be loaded. The third input is a True/False option (verbose, default to False) to decide whether the function should print progress messages (never used in code).
2. It starts by creating an empty list where it will store all the processed images.
3. It loops over all the small folders inside the main folder path. Each small folder usually represents a different person.
4. For every small folder, it loops through every picture inside it. It builds the full file path for each picture.
5. It reads the image file from the computer. At this point, the image is loaded as it is (in color).
6. It converts the picture into a black-and-white (grayscale) image. This makes the images simpler and helps the model learn faster.
7. It resizes the picture to a fixed size: 105×105 pixels. This ensures all images have exactly the same shape.
8. It reshapes the image to add an extra dimension. This step helps the ML model understand the format correctly.
9. It adds the processed image to the list.
10. Once all pictures have been processed, it stacks them together into a single large array and returns it to caller.

In simple words, this function walks through all folders, reads every picture, turns it into a neat black-and-white, same-size version, and collects them together so they are ready for the model to use.

2. **get_triplet()** function (5 Marks) - Write your answer here:

The `get_triplet()` function randomly picks two positions (indexes) from a given range, which is passed in through the shape parameter. The first random index is used for both the anchor and the positive image/s (so they point to the same image/s), while the second random

index is used for the negative image/s (image/s from different person). It then returns these three indexes as a tuple to its caller. These three indexes are later used during training as a triplet (anchor, positive, negative) so that the model can learn to tell apart images that should match from images that should not match.

Step by Step Explanation

1. The function takes one input called shape and it tells the function how many images are available in the dataset. (For example, if there are 500 images, shape[0] will be 500.)
2. It randomly picks a number for the first person (class_1). This random number is used to choose both the anchor and the positive images from the same person.
3. It randomly picks another number for the second person (class_2). This number is used to choose the negative image. The function makes sure this number is not equal to class_1, so that the negative image is always from a different person.
4. It assigns three values:
 - a = class_1 → this will be the anchor
 - p = class_1 → this will be the positive (same person as anchor)
 - n = class_2 → this will be the negative (different person)
5. Finally, it returns these three values as a triplet of (anchor, positive, negative) indexes.

In short, this function randomly picks two image positions from the dataset, one for a person (used twice as anchor and positive) and another for a different person (negative), so the training process can learn how to compare faces.

3. get_triplet_batch() function (5 Marks) - Write your answer here

The get_triplet_batch() function creates a whole group (batch) of triplets. Each triplet contains three images: an anchor, a positive (same person), and a negative (different person). It repeats this process as many times as needed and then returns three separate sets of images. These batches are later fed into the model during training so the model can learn how to compare faces.

Step by Step Explanation

1. It takes two inputs, a batch_size that tells how many triplets to create and a boolean train_data (default to True) to decide whether to use training images or testing images.
2. It creates three empty lists to store the anchor images, positive images, and negative images for the batch.
 - It chooses which dataset to use.
 - If train_data is True, it uses the training images.
 - If False, it uses the validation images.

3. It runs a loop `batch_size` times. Each loop creates one triplet.
4. Inside the loop, it calls `get_triplet()` to get three random indexes (as described above) one for the anchor, one for the positive, and one for the negative.
5. It uses these indexes to pick the actual images from the dataset and adds them to their respective lists.
6. After the loop finishes, it converts all three lists (anchors, positives, negatives) into numpy arrays.
7. Finally, it returns these three arrays so the training or testing process can use them as a batch of triplets.

```
In [6]: # calling the Preprocessing class  
preprocess_data = PreProcessing(verbose=True)
```

The code above is creating an instance of the `PreProcessing` class and setting the `verbose` parameter to `True`. The `PreProcessing` class is a custom class that contains methods for preprocessing data. The `verbose` parameter is used to control whether or not the class will print out information about the steps it is taking during the preprocessing process. Setting it to `True` means that the class will print out information, while setting it to `False` means that it will not. The variable `preprocess_data` now holds an instance of the `PreProcessing` class with the `verbose` parameter set to `True`.

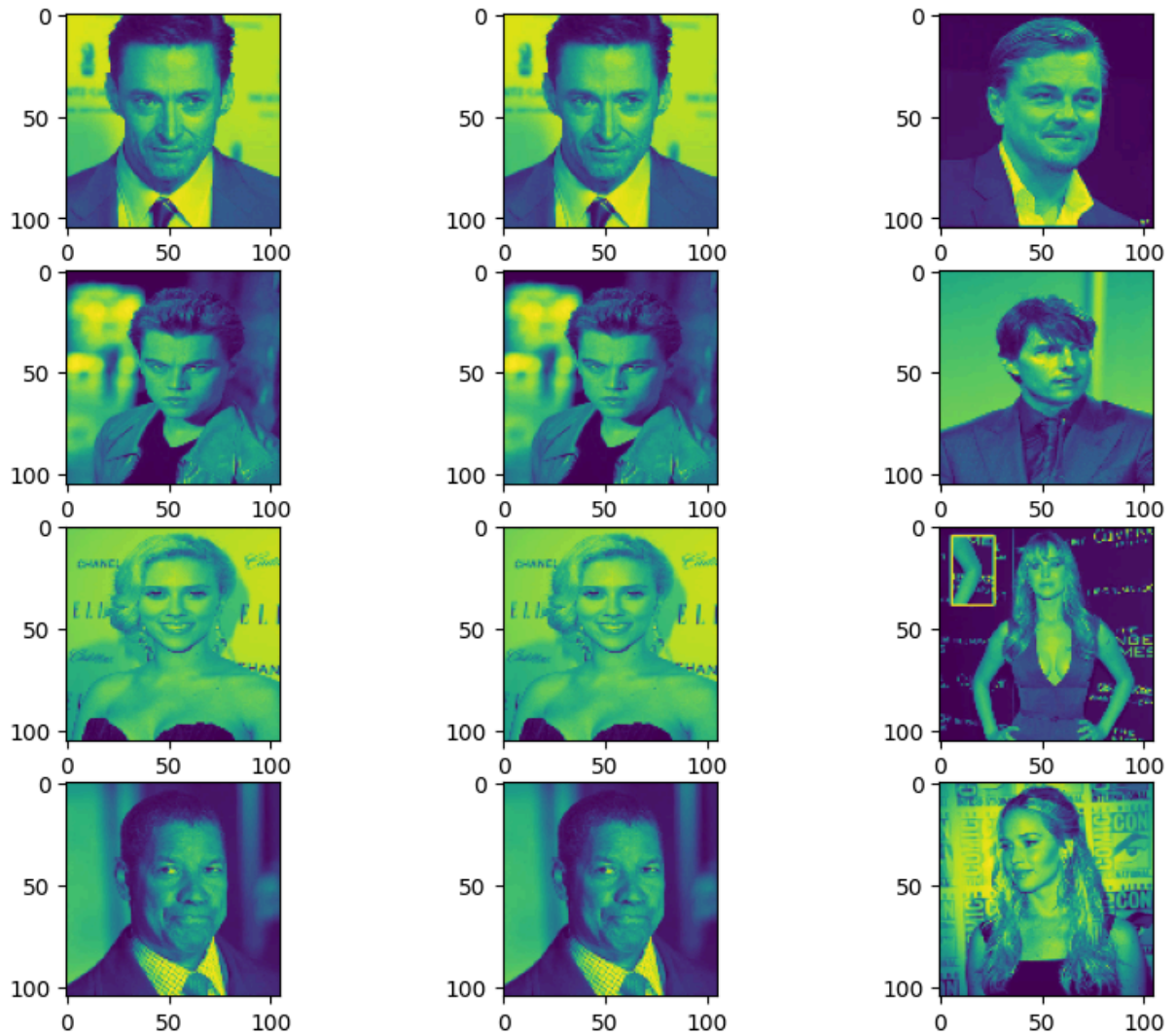
```
In [7]: # calling the get_triplet_batch function with a batch_size of 4 and setting the tra  
batch_size = 4  
anchor_image, positive_image, negative_image = preprocess_data.get_triplet_batch(ba
```

```
In [8]: # checking the shape of a loaded image with size 4  
anchor_image.shape
```

```
Out[8]: (4, 105, 105, 1)
```

2.4.3 Plot the images in the dataset by completing the code in the following cell (5 Marks)

```
In [9]: # plotting the images read using the get_triplet_batch function from the preprocess  
fig, ax = plt.subplots(nrows=batch_size, ncols=3, figsize=(10, 8))  
for i in range(batch_size):  
    ax[i, 0].imshow(np.reshape(anchor_image[i], (105, 105)))  
    ax[i, 1].imshow(np.reshape(positive_image[i], (105, 105)))  
    ax[i, 2].imshow(np.reshape(negative_image[i], (105, 105)))  
plt.show()
```



```
In [10]: cv2_imshow(np.reshape(negative_image[i], (105, 105)))
```



- The images read using cv2 and plotted using matplotlib may appear different in color because the two libraries use different color models and color spaces.
- cv2 uses the BGR (Blue, Green, Red) color model which is native to most cameras and image capture devices. Matplotlib, on the other hand, uses the RGB (Red, Green, Blue) color model which is the standard color model used in most computer displays.
- Additionally, cv2 uses the sRGB color space which is designed to match the way that the human eye perceives color while matplotlib uses the RGB color space which is mainly used in digital imaging.

- Therefore, when an image is read using cv2 and then plotted using matplotlib, the color model and color space used by the two libraries are different, which can cause the image to appear different in color.

3. Model Engineering (20 Marks)

3.1 Building the base_cnn/embedding model (10 Marks)

```
In [11]: emb_size = 128
emb_model = Sequential()

### Complete the following code by removing the blank space and completing the code
emb_model.add(Conv2D(64, (10,10), activation='relu', input_shape=(105, 105, 1)))

emb_model.add(BatchNormalization())

###Add a maxpooling2D Layer
emb_model.add(MaxPooling2D())

emb_model.add(Dropout(0.5))
emb_model.add(Conv2D(32, (7,7), activation='relu'))
emb_model.add(BatchNormalization())
emb_model.add(MaxPooling2D())
emb_model.add(Dropout(0.5))
emb_model.add(Conv2D(64, (4,4), activation='relu'))
emb_model.add(BatchNormalization())
### Add a flatten Layer
emb_model.add(Flatten())

### add the correct activation function for the Dense Layer
emb_model.add(Dense(emb_size, activation='sigmoid'))
emb_model.add(Lambda(lambda x:tf.keras.backend.l2_normalize(x, axis=1)))
```

3.2 Answer the following questions

3.2.1 What is the use of the sequential layer in the model designed above?

Write your answer here:

The Sequential layer is used to build the model step by step, one layer after another, in a straight line. It works like stacking building blocks: you add a convolution layer, then a pooling layer, then another convolution layer, and so on. Each new layer directly takes the output of the previous layer. This makes the model simple to create and easy to read because everything flows in one direction from start to end without any branching or mixing.

3.2.2 What is the use of the lambda layer in the function defined above?

Write your answer here:

The Lambda layer does a quick normalization of the numbers that come out of the model. You can think of the model's output as a long list of numbers or embedding that describe a face. Sometimes this list is large, and sometimes it's smaller. The Lambda layer fixes this by adjusting the numbers so the whole list has a length of exactly 1. This process is called L2 normalization.

This helps the model compare faces more fairly. It won't get confused by lists that are too big or too small. Instead, it only pays attention to the pattern or shape of the numbers, which is what really matters when deciding if two faces look alike. In short, the Lambda layer cleans and normalizes the face numbers so the model can compare them correctly and learn better.

- The first line sets the variable "emb_size" to 128. This variable is used later in the code to specify the size of the embedding output.
- The third line adds a 2D convolutional layer to the model with 64 filters, a kernel size of (10,10) and a ReLU activation function. The input shape parameter specifies that the input to this layer will have a shape of (105,105,1), likely indicating that the input images are 105x105 pixels with 1 color channel.
- The fourth line adds a batch normalization layer to the model. This layer normalizes the outputs of the previous layer, helping to improve the stability and performance of the model.
- The fifth line adds a max pooling layer to the model. This layer reduces the spatial dimensionality of the previous layer's output by taking the maximum value of each pooling window.
- The sixth line adds a dropout layer to the model with a dropout rate of 0.5. This layer randomly "drops out" a certain percentage of the previous layer's output, helping to prevent overfitting.
- The seventh line is similar to the third line, it adds a 2D convolutional layer with 32 filters, a kernel size of (7,7) and a ReLU activation function.
- The eighth line is similar to the fourth line, it adds a batch normalization layer to the model.
- The ninth line is similar to the fifth line, it adds a max pooling layer to the model.
- The tenth line is similar to the sixth line, it adds a dropout layer to the model with a dropout rate of 0.5.

- The eleventh line is similar to the third line, it adds a 2D convolutional layer with 64 filters, a kernel size of (4,4) and a ReLU activation function.
- The twelfth line is similar to the fourth line, it adds a batch normalization layer to the model.
- The thirteenth line adds a flatten layer to the model. This layer converts the previous layer's output from a multi-dimensional array to a single dimension array.
- The fourteenth line adds a dense layer to the model with emb_size units and a sigmoid activation function. This layer will output the embeddings with the size defined in emb_size variable.

In [12]: `emb_model.summary()`

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 96, 96, 64)	6,464
batch_normalization (BatchNormalization)	(None, 96, 96, 64)	256
max_pooling2d (MaxPooling2D)	(None, 48, 48, 64)	0
dropout (Dropout)	(None, 48, 48, 64)	0
conv2d_1 (Conv2D)	(None, 42, 42, 32)	100,384
batch_normalization_1 (BatchNormalization)	(None, 42, 42, 32)	128
max_pooling2d_1 (MaxPooling2D)	(None, 21, 21, 32)	0
dropout_1 (Dropout)	(None, 21, 21, 32)	0
conv2d_2 (Conv2D)	(None, 18, 18, 64)	32,832
batch_normalization_2 (BatchNormalization)	(None, 18, 18, 64)	256
flatten (Flatten)	(None, 20736)	0
dense (Dense)	(None, 128)	2,654,336
lambda (Lambda)	(None, 128)	0

Total params: 2,794,656 (10.66 MB)

Trainable params: 2,794,336 (10.66 MB)

Non-trainable params: 320 (1.25 KB)

4. Building the Siamese Network with input and output (5 Marks)

Complete the code below by removing the blank spaces

```
In [13]: input_anc = Input(shape=(105,105,1))
input_pos = Input(shape=(105,105,1))
input_neg = Input(shape=(105,105,1))

emb_anc = emb_model(input_anc)
emb_pos = emb_model(input_pos)
emb_neg = emb_model(input_neg)

output = concatenate([emb_anc, emb_pos, emb_neg], axis=1)
### build the Model
siamese_network = Model([input_anc, input_pos, input_neg],output)
```

- The above code starts by defining three inputs for the model: `in_anc`, `in_pos`, and `in_neg`. These inputs have a shape of (105,105,1), which means they are 105x105 images with 1 channel (grayscale).
- The next step is to pass each of the inputs through an existing model called "embedding_model". The output of this model is stored in variables called `em_anc`, `em_pos`, and `em_neg`.
- The outputs from the embedding_model (`em_anc`, `em_pos`, and `em_neg`) are then concatenated together along the first axis (`axis=1`) using the function `concatenate`. This creates a single output that combines the three outputs from the embedding_model.
- Finally, the `Model` class is used to define a new model called "siamese_net" which takes the three inputs (`in_anc`, `in_pos`, and `in_neg`) and produces the concatenated output as its output. This new model is now ready to be trained and used for tasks such as image comparison or similarity learning.

```
In [14]: siamese_network.summary()
```

Model: "functional_13"

Layer (type)	Output Shape	Param #	Connected to
input_layer_1 (InputLayer)	(None, 105, 105, 1)	0	-
input_layer_2 (InputLayer)	(None, 105, 105, 1)	0	-
input_layer_3 (InputLayer)	(None, 105, 105, 1)	0	-
sequential (Sequential)	(None, 128)	2,794,656	input_layer_1[0]... input_layer_2[0]... input_layer_3[0]...
concatenate (Concatenate)	(None, 384)	0	sequential[0][0], sequential[1][0], sequential[2][0]

Total params: 2,794,656 (10.66 MB)

Trainable params: 2,794,336 (10.66 MB)

Non-trainable params: 320 (1.25 KB)

5. Defining the Loss functions (5 Marks)

Complete the code below by removing the blank space to define Triplet Loss function for L2 Distance

```
In [15]: # L2 Distance
def triplet_loss(alpha, emb_dim, cosine=False):
    def loss(y_true, y_pred):
        anc, pos, neg = y_pred[:, :emb_size], y_pred[:, emb_size:2*emb_size], y_pred[
        distance1 = tf.sqrt(tf.reduce_sum(tf.pow(anc - pos, 2), 1, keepdims=True))
        distance2 = tf.sqrt(tf.reduce_sum(tf.pow(anc - neg, 2), 1, keepdims=True))
        return tf.reduce_mean(tf.maximum(distance1 - distance2 + alpha, 0.))
    return loss
```

- The above code defines a function named "triplet_loss" which takes in two parameters, "alpha" and "emb_dim".
- The function also has an optional parameter "cosine" which is set to False by default.
- Within the "triplet_loss" function, there is another nested function named "loss". This function takes in two parameters, "y_true" and "y_pred".
- The code then splits the "y_pred" parameter into three variables: "anc", "pos", and "neg". These variables represent the anchor, positive, and negative embeddings respectively.
- The code then calculates the L2 distance between the anchor and positive embeddings using the formula: $\text{distance1} = \sqrt{\sum((\text{anc} - \text{pos})^2)}$

- The code then calculates the L2 distance between the anchor and negative embeddings using the formula: $\text{distance2} = \sqrt{\sum((\text{anc} - \text{neg})^2)}$
- The code then finds the maximum value between $(\text{distance1} - \text{distance2} + \alpha)$ and 0 using the `tf.maximum` function.
- The code then calculates the mean of the maximum values found in the previous step using the `tf.reduce_mean` function.
- The "loss" function returns the mean value calculated in the previous step.
- The "triplet_loss" function returns the "loss" function.

Triplet loss function for Cosine Distance

```
In [16]: # Cosine Similarity
def triplet_loss(alpha, emb_dim, cosine=True):
    def loss(y_true, y_pred):
        anc, pos, neg = y_pred[:, :emb_size], y_pred[:, emb_size:2*emb_size], y_pred[
        distance1 = tf.keras.losses.cosine_similarity(anc, pos)
        distance2 = tf.keras.losses.cosine_similarity(anc, neg)
        return tf.keras.backend.clip(distance1 - distance2 + alpha, 0., None)
    return loss
```

- The above function "triplet_loss" is defined and takes in three parameters: alpha, emb_dim, and cosine.
- Within "triplet_loss", a nested function "loss" is defined, which takes in two parameters: y_true and y_pred.
- Within "loss", three variables are defined: anc, pos, and neg, which represent the anchor, positive, and negative embeddings respectively.
- These are obtained by slicing the y_pred tensor at specific indices.
- The cosine similarity between the anchor and positive embeddings is calculated and stored in the variable "distance1" using the cosine_similarity function from the `tf.keras.losses` module.
- The cosine similarity between the anchor and negative embeddings is calculated and stored in the variable "distance2" using the cosine_similarity function from the `tf.keras.losses` module.
- The triplet loss is calculated as the difference between "distance1" and "distance2" plus alpha, and is clipped to a minimum value of 0 using the clip function from the `tf.keras.backend` module.
- The final value of the triplet loss is returned.
- Finally, the function "loss" is returned as the output of "triplet_loss".

```
In [17]: # using data generator to yield input and output for the model to fit
def data_generator(batch_size, emb_size):
    while True:
        x = preprocess_data.get_triplet_batch(batch_size)
        y = np.zeros((batch_size, 3*emb_size))
        yield x,y
```

- The function "data_generator" is defined with two input parameters: "batch_size" and "emb_size".
- A while loop is initiated with the condition "True", which means it will continue to execute indefinitely.
- Within the while loop, a variable "x" is assigned the output of the function "get_triplet_batch" from the "preprocess_data" module, with the input of "batch_size".
- A variable "y" is created and assigned an array of zeros with dimensions "batch_size" by "3*emb_size".
- The function yields both variables "x" and "y" as output.

6. Compiling the model with Triplet Loss (5 Marks)

```
In [18]: batch_size = 128
epochs = 10
# In this line the learning_rate param was defined as lr which was throwing an error
opt = tf.keras.optimizers.Adam(learning_rate = 0.0001)
steps_per_epoch = 100
siamese_network.compile(loss=triplet_loss(alpha=0.2, emb_dim=emb_size, cosine=False))
siamese_network.summary()
```

Model: "functional_13"

Layer (type)	Output Shape	Param #	Connected to
input_layer_1 (InputLayer)	(None, 105, 105, 1)	0	-
input_layer_2 (InputLayer)	(None, 105, 105, 1)	0	-
input_layer_3 (InputLayer)	(None, 105, 105, 1)	0	-
sequential (Sequential)	(None, 128)	2,794,656	input_layer_1[0]... input_layer_2[0]... input_layer_3[0]...
concatenate (Concatenate)	(None, 384)	0	sequential[0][0], sequential[1][0], sequential[2][0]

Total params: 2,794,656 (10.66 MB)

Trainable params: 2,794,336 (10.66 MB)

Non-trainable params: 320 (1.25 KB)

7. Fitting the model (5 Marks)

Fit the siamese network by completing the code below:

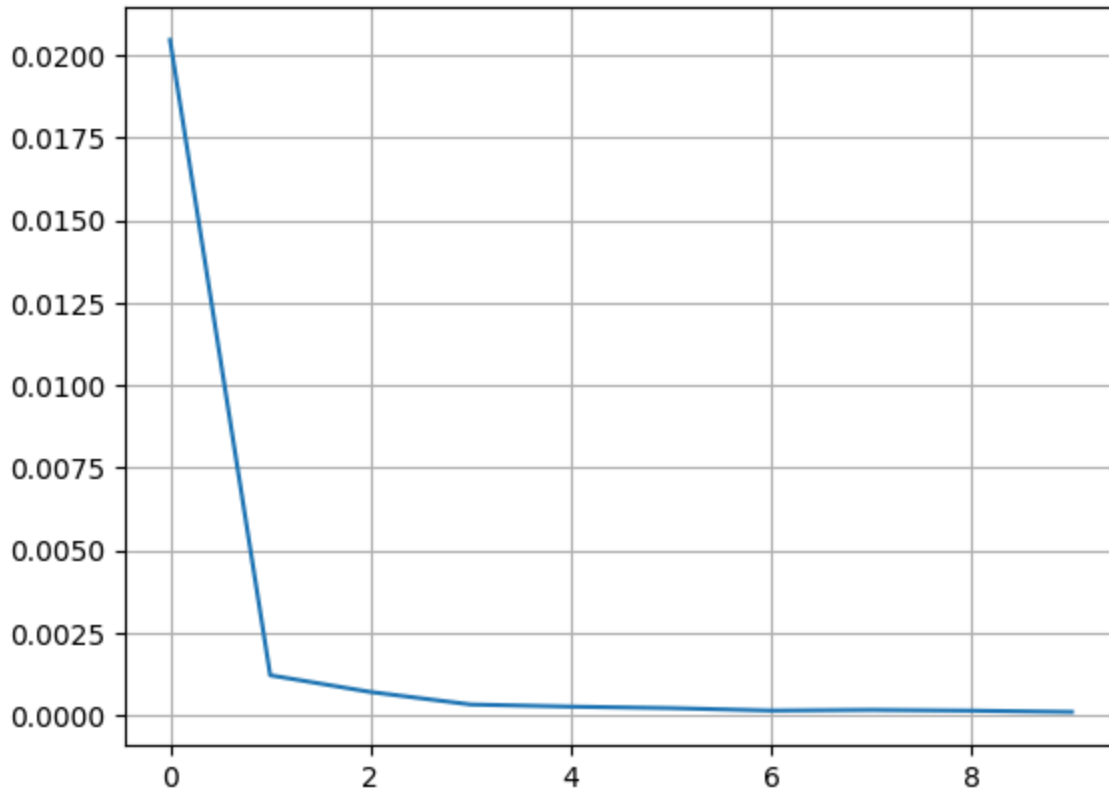
```
In [19]: history = siamese_network.fit(  
    data_generator(batch_size, emb_size),  
    steps_per_epoch=steps_per_epoch,  
    epochs=epochs  
)
```

```
Epoch 1/10  
100/100 ————— 34s 165ms/step - loss: 0.0492  
Epoch 2/10  
100/100 ————— 17s 166ms/step - loss: 0.0015  
Epoch 3/10  
100/100 ————— 17s 166ms/step - loss: 7.5496e-04  
Epoch 4/10  
100/100 ————— 17s 167ms/step - loss: 3.7569e-04  
Epoch 5/10  
100/100 ————— 17s 168ms/step - loss: 3.2554e-04  
Epoch 6/10  
100/100 ————— 17s 169ms/step - loss: 1.8543e-04  
Epoch 7/10  
100/100 ————— 17s 169ms/step - loss: 1.4432e-04  
Epoch 8/10  
100/100 ————— 17s 169ms/step - loss: 1.5469e-04  
Epoch 9/10  
100/100 ————— 17s 169ms/step - loss: 1.8941e-04  
Epoch 10/10  
100/100 ————— 17s 168ms/step - loss: 1.2692e-04
```

8. Model Evaluation

8.1 Plotting the Loss curve (5 Marks)

```
In [20]: plt.plot(history.history['loss'], label='Loss')  
plt.grid(True)  
plt.show()
```



```
In [21]: print(history.history['loss'])
```

```
[0.020467815920710564, 0.0012087835930287838, 0.0007023646030575037, 0.0003230260917
916894, 0.0002581655280664563, 0.000212843922781758, 0.00013891674461774528, 0.00015
864732267800719, 0.00013543499517254531, 9.71310946624726e-05]
```

8.2 Obtaining train and validation accuracies (5 Marks)

```
In [22]: test_sample = 150
nway_min = 5
nway_max = 25
test_acc = []
nway_list = []
for nway in range(nway_min, nway_max+1,5):
    print("Checking %d way accuracy..." % nway)
    correct_count = 0
    for _ in range(test_sample):
        nway_anchor, nway_positive, _ = preprocess_data.get_triplet_batch(nway, train_d
        nway_anchor_embedding = emb_model.predict(nway_anchor)
        sample_index = random.randint(0,nway-1)
        sample_embedding = emb_model.predict(np.expand_dims(nway_positive[sample_index]
        distance = tf.sqrt(tf.reduce_sum(tf.pow(nway_anchor_embedding - sample_embeddin
        if np.argmin(distance) == sample_index:
            correct_count+=1
    nway_list.append(nway)
    acc = (correct_count/test_sample)*100.
    test_acc.append(acc)
    print("Accuracy %.2f" % acc)
```

Checking 5 way accuracy....

1/1	=====	1s	803ms/step
1/1	=====	1s	641ms/step
1/1	=====	0s	29ms/step
1/1	=====	0s	28ms/step
1/1	=====	0s	28ms/step
1/1	=====	0s	28ms/step
1/1	=====	0s	28ms/step
1/1	=====	0s	28ms/step
1/1	=====	0s	27ms/step
1/1	=====	0s	28ms/step
1/1	=====	0s	29ms/step
1/1	=====	0s	29ms/step
1/1	=====	0s	30ms/step
1/1	=====	0s	28ms/step
1/1	=====	0s	27ms/step
1/1	=====	0s	40ms/step
1/1	=====	0s	31ms/step
1/1	=====	0s	29ms/step
1/1	=====	0s	28ms/step
1/1	=====	0s	28ms/step
1/1	=====	0s	29ms/step
1/1	=====	0s	28ms/step
1/1	=====	0s	27ms/step
1/1	=====	0s	29ms/step
1/1	=====	0s	27ms/step
1/1	=====	0s	28ms/step
1/1	=====	0s	27ms/step
1/1	=====	0s	27ms/step
1/1	=====	0s	28ms/step
1/1	=====	0s	30ms/step
1/1	=====	0s	29ms/step
1/1	=====	0s	28ms/step
1/1	=====	0s	40ms/step
1/1	=====	0s	29ms/step
1/1	=====	0s	28ms/step
1/1	=====	0s	29ms/step
1/1	=====	0s	28ms/step
1/1	=====	0s	29ms/step
1/1	=====	0s	28ms/step
1/1	=====	0s	28ms/step
1/1	=====	0s	27ms/step
1/1	=====	0s	31ms/step
1/1	=====	0s	27ms/step
1/1	=====	0s	28ms/step
1/1	=====	0s	28ms/step
1/1	=====	0s	29ms/step
1/1	=====	0s	28ms/step
1/1	=====	0s	28ms/step
1/1	=====	0s	27ms/step
1/1	=====	0s	38ms/step
1/1	=====	0s	27ms/step
1/1	=====	0s	28ms/step
1/1	=====	0s	27ms/step
1/1	=====	0s	28ms/step

1/1	—————	0s	28ms/step
1/1	—————	0s	30ms/step
1/1	—————	0s	27ms/step
1/1	—————	0s	28ms/step
1/1	—————	0s	28ms/step
1/1	—————	0s	28ms/step
1/1	—————	0s	28ms/step
1/1	—————	0s	28ms/step
1/1	—————	0s	28ms/step
1/1	—————	0s	27ms/step
1/1	—————	0s	28ms/step
1/1	—————	0s	37ms/step
1/1	—————	0s	30ms/step
1/1	—————	0s	29ms/step
1/1	—————	0s	29ms/step
1/1	—————	0s	30ms/step
1/1	—————	0s	29ms/step
1/1	—————	0s	27ms/step
1/1	—————	0s	28ms/step
1/1	—————	0s	27ms/step
1/1	—————	0s	28ms/step
1/1	—————	0s	27ms/step
1/1	—————	0s	29ms/step
1/1	—————	0s	29ms/step
1/1	—————	0s	29ms/step
1/1	—————	0s	27ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	30ms/step
1/1	—————	0s	30ms/step
1/1	—————	0s	38ms/step
1/1	—————	0s	30ms/step
1/1	—————	0s	28ms/step
1/1	—————	0s	28ms/step
1/1	—————	0s	30ms/step
1/1	—————	0s	40ms/step
1/1	—————	0s	45ms/step
1/1	—————	0s	43ms/step
1/1	—————	0s	46ms/step
1/1	—————	0s	43ms/step
1/1	—————	0s	43ms/step
1/1	—————	0s	44ms/step
1/1	—————	0s	41ms/step
1/1	—————	0s	40ms/step
1/1	—————	0s	43ms/step
1/1	—————	0s	39ms/step
1/1	—————	0s	41ms/step
1/1	—————	0s	40ms/step
1/1	—————	0s	50ms/step
1/1	—————	0s	43ms/step
1/1	—————	0s	43ms/step
1/1	—————	0s	40ms/step
1/1	—————	0s	41ms/step
1/1	—————	0s	41ms/step
1/1	—————	0s	39ms/step

1/1	_____	0s	40ms/step
1/1	_____	0s	39ms/step
1/1	_____	0s	48ms/step
1/1	_____	0s	45ms/step
1/1	_____	0s	50ms/step
1/1	_____	0s	43ms/step
1/1	_____	0s	46ms/step
1/1	_____	0s	49ms/step
1/1	_____	0s	48ms/step
1/1	_____	0s	46ms/step
1/1	_____	0s	51ms/step
1/1	_____	0s	46ms/step
1/1	_____	0s	28ms/step
1/1	_____	0s	31ms/step
1/1	_____	0s	28ms/step
1/1	_____	0s	30ms/step
1/1	_____	0s	30ms/step
1/1	_____	0s	29ms/step
1/1	_____	0s	29ms/step
1/1	_____	0s	30ms/step
1/1	_____	0s	35ms/step
1/1	_____	0s	30ms/step
1/1	_____	0s	28ms/step
1/1	_____	0s	28ms/step
1/1	_____	0s	27ms/step
1/1	_____	0s	29ms/step
1/1	_____	0s	30ms/step
1/1	_____	0s	30ms/step
1/1	_____	0s	28ms/step
1/1	_____	0s	29ms/step
1/1	_____	0s	28ms/step
1/1	_____	0s	29ms/step
1/1	_____	0s	28ms/step
1/1	_____	0s	28ms/step
1/1	_____	0s	29ms/step
1/1	_____	0s	30ms/step
1/1	_____	0s	27ms/step
1/1	_____	0s	31ms/step
1/1	_____	0s	28ms/step
1/1	_____	0s	34ms/step
1/1	_____	0s	30ms/step
1/1	_____	0s	32ms/step
1/1	_____	0s	35ms/step
1/1	_____	0s	31ms/step
1/1	_____	0s	30ms/step
1/1	_____	0s	30ms/step
1/1	_____	0s	28ms/step
1/1	_____	0s	29ms/step
1/1	_____	0s	29ms/step
1/1	_____	0s	28ms/step
1/1	_____	0s	30ms/step
1/1	_____	0s	33ms/step
1/1	_____	0s	31ms/step
1/1	_____	0s	32ms/step
1/1	_____	0s	29ms/step
1/1	_____	0s	32ms/step

1/1	0s	29ms/step
1/1	0s	29ms/step
1/1	0s	29ms/step
1/1	0s	31ms/step
1/1	0s	29ms/step
1/1	0s	29ms/step
1/1	0s	28ms/step
1/1	0s	30ms/step
1/1	0s	29ms/step
1/1	0s	29ms/step
1/1	0s	29ms/step
1/1	0s	30ms/step
1/1	0s	30ms/step
1/1	0s	37ms/step
1/1	0s	28ms/step
1/1	0s	30ms/step
1/1	0s	29ms/step
1/1	0s	28ms/step
1/1	0s	29ms/step
1/1	0s	31ms/step
1/1	0s	32ms/step
1/1	0s	30ms/step
1/1	0s	42ms/step
1/1	0s	32ms/step
1/1	0s	31ms/step
1/1	0s	29ms/step
1/1	0s	28ms/step
1/1	0s	29ms/step
1/1	0s	30ms/step
1/1	0s	53ms/step
1/1	0s	29ms/step
1/1	0s	31ms/step
1/1	0s	32ms/step
1/1	0s	32ms/step
1/1	0s	28ms/step
1/1	0s	30ms/step
1/1	0s	30ms/step
1/1	0s	29ms/step
1/1	0s	29ms/step
1/1	0s	29ms/step
1/1	0s	29ms/step
1/1	0s	29ms/step
1/1	0s	31ms/step
1/1	0s	30ms/step
1/1	0s	36ms/step
1/1	0s	29ms/step
1/1	0s	30ms/step
1/1	0s	31ms/step
1/1	0s	29ms/step
1/1	0s	28ms/step
1/1	0s	29ms/step
1/1	0s	30ms/step
1/1	0s	30ms/step
1/1	0s	27ms/step
1/1	0s	30ms/step

1/1		0s	29ms/step
1/1		0s	32ms/step
1/1		0s	29ms/step
1/1		0s	30ms/step
1/1		0s	28ms/step
1/1		0s	31ms/step
1/1		0s	28ms/step
1/1		0s	29ms/step
1/1		0s	30ms/step
1/1		0s	31ms/step
1/1		0s	28ms/step
1/1		0s	30ms/step
1/1		0s	30ms/step
1/1		0s	31ms/step
1/1		0s	29ms/step
1/1		0s	32ms/step
1/1		0s	29ms/step
1/1		0s	32ms/step
1/1		0s	28ms/step
1/1		0s	29ms/step
1/1		0s	28ms/step
1/1		0s	40ms/step
1/1		0s	30ms/step
1/1		0s	31ms/step
1/1		0s	30ms/step
1/1		0s	32ms/step
1/1		0s	29ms/step
1/1		0s	32ms/step
1/1		0s	31ms/step
1/1		0s	29ms/step
1/1		0s	28ms/step
1/1		0s	31ms/step
1/1		0s	29ms/step
1/1		0s	32ms/step
1/1		0s	29ms/step
1/1		0s	31ms/step
1/1		0s	32ms/step
1/1		0s	41ms/step
1/1		0s	29ms/step
1/1		0s	34ms/step
1/1		0s	30ms/step
1/1		0s	31ms/step
1/1		0s	29ms/step
1/1		0s	29ms/step
1/1		0s	29ms/step
1/1		0s	30ms/step
1/1		0s	30ms/step
1/1		0s	31ms/step
1/1		0s	33ms/step
1/1		0s	33ms/step
1/1		0s	30ms/step
1/1		0s	31ms/step
1/1		0s	40ms/step
1/1		0s	31ms/step
1/1		0s	40ms/step
1/1		0s	47ms/step

1/1	=====	0s	43ms/step
1/1	=====	0s	42ms/step
1/1	=====	0s	39ms/step
1/1	=====	0s	41ms/step
1/1	=====	0s	43ms/step
1/1	=====	0s	40ms/step
1/1	=====	0s	60ms/step
1/1	=====	0s	43ms/step
1/1	=====	0s	45ms/step
1/1	=====	0s	41ms/step
1/1	=====	0s	62ms/step
1/1	=====	0s	47ms/step
1/1	=====	0s	43ms/step
1/1	=====	0s	48ms/step
1/1	=====	0s	44ms/step
1/1	=====	0s	44ms/step
1/1	=====	0s	43ms/step
1/1	=====	0s	49ms/step
1/1	=====	0s	45ms/step
1/1	=====	0s	48ms/step
1/1	=====	0s	49ms/step

Accuracy 100.00

Checking 10 way accuracy....

1/1	=====	1s	886ms/step
1/1	=====	0s	41ms/step
1/1	=====	0s	30ms/step
1/1	=====	0s	27ms/step
1/1	=====	0s	30ms/step
1/1	=====	0s	29ms/step
1/1	=====	0s	32ms/step
1/1	=====	0s	30ms/step
1/1	=====	0s	32ms/step
1/1	=====	0s	31ms/step
1/1	=====	0s	29ms/step
1/1	=====	0s	28ms/step
1/1	=====	0s	31ms/step
1/1	=====	0s	30ms/step
1/1	=====	0s	30ms/step
1/1	=====	0s	28ms/step
1/1	=====	0s	29ms/step
1/1	=====	0s	29ms/step
1/1	=====	0s	33ms/step
1/1	=====	0s	31ms/step
1/1	=====	0s	32ms/step
1/1	=====	0s	44ms/step
1/1	=====	0s	32ms/step
1/1	=====	0s	42ms/step
1/1	=====	0s	32ms/step
1/1	=====	0s	34ms/step
1/1	=====	0s	30ms/step
1/1	=====	0s	29ms/step
1/1	=====	0s	31ms/step
1/1	=====	0s	30ms/step
1/1	=====	0s	32ms/step
1/1	=====	0s	31ms/step
1/1	=====	0s	31ms/step

1/1		0s	39ms/step
1/1		0s	31ms/step
1/1		0s	30ms/step
1/1		0s	35ms/step
1/1		0s	30ms/step
1/1		0s	33ms/step
1/1		0s	30ms/step
1/1		0s	29ms/step
1/1		0s	29ms/step
1/1		0s	32ms/step
1/1		0s	30ms/step
1/1		0s	31ms/step
1/1		0s	31ms/step
1/1		0s	31ms/step
1/1		0s	29ms/step
1/1		0s	32ms/step
1/1		0s	30ms/step
1/1		0s	30ms/step
1/1		0s	30ms/step
1/1		0s	30ms/step
1/1		0s	33ms/step
1/1		0s	32ms/step
1/1		0s	33ms/step
1/1		0s	33ms/step
1/1		0s	33ms/step
1/1		0s	31ms/step
1/1		0s	29ms/step
1/1		0s	31ms/step
1/1		0s	30ms/step
1/1		0s	30ms/step
1/1		0s	30ms/step
1/1		0s	30ms/step
1/1		0s	29ms/step
1/1		0s	31ms/step
1/1		0s	32ms/step
1/1		0s	34ms/step
1/1		0s	31ms/step
1/1		0s	32ms/step
1/1		0s	29ms/step
1/1		0s	30ms/step
1/1		0s	31ms/step
1/1		0s	30ms/step
1/1		0s	31ms/step
1/1		0s	31ms/step
1/1		0s	31ms/step
1/1		0s	32ms/step
1/1		0s	31ms/step
1/1		0s	35ms/step
1/1		0s	31ms/step
1/1		0s	32ms/step
1/1		0s	36ms/step
1/1		0s	31ms/step
1/1		0s	29ms/step
1/1		0s	30ms/step
1/1		0s	30ms/step
1/1		0s	30ms/step

1/1		0s	28ms/step
1/1		0s	31ms/step
1/1		0s	31ms/step
1/1		0s	33ms/step
1/1		0s	31ms/step
1/1		0s	33ms/step
1/1		0s	32ms/step
1/1		0s	31ms/step
1/1		0s	30ms/step
1/1		0s	31ms/step
1/1		0s	30ms/step
1/1		0s	31ms/step
1/1		0s	28ms/step
1/1		0s	33ms/step
1/1		0s	33ms/step
1/1		0s	35ms/step
1/1		0s	35ms/step
1/1		0s	32ms/step
1/1		0s	30ms/step
1/1		0s	32ms/step
1/1		0s	30ms/step
1/1		0s	33ms/step
1/1		0s	29ms/step
1/1		0s	30ms/step
1/1		0s	30ms/step
1/1		0s	44ms/step
1/1		0s	31ms/step
1/1		0s	34ms/step
1/1		0s	30ms/step
1/1		0s	34ms/step
1/1		0s	30ms/step
1/1		0s	35ms/step
1/1		0s	29ms/step
1/1		0s	30ms/step
1/1		0s	29ms/step
1/1		0s	32ms/step
1/1		0s	29ms/step
1/1		0s	32ms/step
1/1		0s	31ms/step
1/1		0s	33ms/step
1/1		0s	32ms/step
1/1		0s	32ms/step
1/1		0s	30ms/step
1/1		0s	33ms/step
1/1		0s	30ms/step
1/1		0s	30ms/step
1/1		0s	30ms/step
1/1		0s	31ms/step
1/1		0s	29ms/step
1/1		0s	33ms/step
1/1		0s	32ms/step
1/1		0s	33ms/step
1/1		0s	31ms/step
1/1		0s	50ms/step
1/1		0s	47ms/step
1/1		0s	44ms/step

1/1	—————	0s	47ms/step
1/1	—————	0s	57ms/step
1/1	—————	0s	44ms/step
1/1	—————	0s	43ms/step
1/1	—————	0s	42ms/step
1/1	—————	0s	47ms/step
1/1	—————	0s	42ms/step
1/1	—————	0s	41ms/step
1/1	—————	0s	45ms/step
1/1	—————	0s	47ms/step
1/1	—————	0s	47ms/step
1/1	—————	0s	46ms/step
1/1	—————	0s	41ms/step
1/1	—————	0s	45ms/step
1/1	—————	0s	43ms/step
1/1	—————	0s	47ms/step
1/1	—————	0s	38ms/step
1/1	—————	0s	52ms/step
1/1	—————	0s	60ms/step
1/1	—————	0s	54ms/step
1/1	—————	0s	42ms/step
1/1	—————	0s	54ms/step
1/1	—————	0s	51ms/step
1/1	—————	0s	46ms/step
1/1	—————	0s	49ms/step
1/1	—————	0s	53ms/step
1/1	—————	0s	30ms/step
1/1	—————	0s	31ms/step
1/1	—————	0s	29ms/step
1/1	—————	0s	42ms/step
1/1	—————	0s	31ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	31ms/step
1/1	—————	0s	30ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	29ms/step
1/1	—————	0s	31ms/step
1/1	—————	0s	30ms/step
1/1	—————	0s	37ms/step
1/1	—————	0s	30ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	45ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	31ms/step
1/1	—————	0s	31ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	30ms/step
1/1	—————	0s	31ms/step
1/1	—————	0s	31ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	31ms/step
1/1	—————	0s	33ms/step

1/1		0s	33ms/step
1/1		0s	36ms/step
1/1		0s	44ms/step
1/1		0s	32ms/step
1/1		0s	30ms/step
1/1		0s	33ms/step
1/1		0s	32ms/step
1/1		0s	32ms/step
1/1		0s	29ms/step
1/1		0s	34ms/step
1/1		0s	34ms/step
1/1		0s	34ms/step
1/1		0s	32ms/step
1/1		0s	35ms/step
1/1		0s	30ms/step
1/1		0s	34ms/step
1/1		0s	30ms/step
1/1		0s	37ms/step
1/1		0s	30ms/step
1/1		0s	30ms/step
1/1		0s	29ms/step
1/1		0s	38ms/step
1/1		0s	32ms/step
1/1		0s	32ms/step
1/1		0s	32ms/step
1/1		0s	34ms/step
1/1		0s	33ms/step
1/1		0s	32ms/step
1/1		0s	33ms/step
1/1		0s	32ms/step
1/1		0s	29ms/step
1/1		0s	30ms/step
1/1		0s	32ms/step
1/1		0s	35ms/step
1/1		0s	31ms/step
1/1		0s	32ms/step
1/1		0s	34ms/step
1/1		0s	33ms/step
1/1		0s	32ms/step
1/1		0s	41ms/step
1/1		0s	33ms/step
1/1		0s	32ms/step
1/1		0s	31ms/step
1/1		0s	32ms/step
1/1		0s	34ms/step
1/1		0s	34ms/step
1/1		0s	32ms/step
1/1		0s	40ms/step
1/1		0s	34ms/step
1/1		0s	33ms/step
1/1		0s	33ms/step
1/1		0s	34ms/step
1/1		0s	33ms/step
1/1		0s	34ms/step
1/1		0s	31ms/step
1/1		0s	32ms/step

1/1	0s	31ms/step
1/1	0s	34ms/step
1/1	0s	35ms/step
1/1	0s	34ms/step
1/1	0s	33ms/step
1/1	0s	36ms/step
1/1	0s	36ms/step
1/1	0s	31ms/step
1/1	0s	35ms/step
1/1	0s	34ms/step
1/1	0s	30ms/step
1/1	0s	33ms/step
1/1	0s	31ms/step
1/1	0s	34ms/step
1/1	0s	33ms/step
1/1	0s	33ms/step
1/1	0s	34ms/step
1/1	0s	34ms/step
1/1	0s	30ms/step
1/1	0s	34ms/step
1/1	0s	31ms/step
1/1	0s	41ms/step
1/1	0s	31ms/step
1/1	0s	33ms/step
1/1	0s	31ms/step
1/1	0s	32ms/step
1/1	0s	31ms/step
1/1	0s	33ms/step
1/1	0s	33ms/step
1/1	0s	34ms/step
1/1	0s	31ms/step
1/1	0s	31ms/step
1/1	0s	32ms/step
1/1	0s	31ms/step
1/1	0s	31ms/step
1/1	0s	32ms/step
1/1	0s	32ms/step
1/1	0s	33ms/step
1/1	0s	31ms/step
1/1	0s	34ms/step
1/1	0s	33ms/step
1/1	0s	33ms/step
1/1	0s	31ms/step

Accuracy 98.67

Checking 15 way accuracy....

1/1	1s	759ms/step
1/1	0s	30ms/step
1/1	0s	31ms/step
1/1	0s	30ms/step
1/1	0s	31ms/step
1/1	0s	38ms/step
1/1	0s	51ms/step
1/1	0s	51ms/step
1/1	0s	51ms/step
1/1	0s	40ms/step
1/1	0s	57ms/step

1/1	—————	0s	40ms/step
1/1	—————	0s	42ms/step
1/1	—————	0s	48ms/step
1/1	—————	0s	43ms/step
1/1	—————	0s	38ms/step
1/1	—————	0s	45ms/step
1/1	—————	0s	41ms/step
1/1	—————	0s	41ms/step
1/1	—————	0s	47ms/step
1/1	—————	0s	52ms/step
1/1	—————	0s	47ms/step
1/1	—————	0s	60ms/step
1/1	—————	0s	39ms/step
1/1	—————	0s	40ms/step
1/1	—————	0s	44ms/step
1/1	—————	0s	50ms/step
1/1	—————	0s	51ms/step
1/1	—————	0s	45ms/step
1/1	—————	0s	46ms/step
1/1	—————	0s	53ms/step
1/1	—————	0s	54ms/step
1/1	—————	0s	49ms/step
1/1	—————	0s	54ms/step
1/1	—————	0s	51ms/step
1/1	—————	0s	52ms/step
1/1	—————	0s	37ms/step
1/1	—————	0s	30ms/step
1/1	—————	0s	31ms/step
1/1	—————	0s	31ms/step
1/1	—————	0s	31ms/step
1/1	—————	0s	29ms/step
1/1	—————	0s	38ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	31ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	31ms/step
1/1	—————	0s	30ms/step
1/1	—————	0s	39ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	37ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	31ms/step
1/1	—————	0s	30ms/step
1/1	—————	0s	34ms/step

1/1	—————	0s	32ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	31ms/step
1/1	—————	0s	31ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	40ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	44ms/step
1/1	—————	0s	37ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	30ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	31ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	38ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	31ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	31ms/step
1/1	—————	0s	44ms/step
1/1	—————	0s	31ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	31ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	50ms/step
1/1	—————	0s	38ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	31ms/step
1/1	—————	0s	32ms/step

1/1		0s	32ms/step
1/1		0s	32ms/step
1/1		0s	31ms/step
1/1		0s	33ms/step
1/1		0s	32ms/step
1/1		0s	36ms/step
1/1		0s	33ms/step
1/1		0s	39ms/step
1/1		0s	34ms/step
1/1		0s	35ms/step
1/1		0s	34ms/step
1/1		0s	33ms/step
1/1		0s	31ms/step
1/1		0s	33ms/step
1/1		0s	32ms/step
1/1		0s	35ms/step
1/1		0s	34ms/step
1/1		0s	36ms/step
1/1		0s	34ms/step
1/1		0s	33ms/step
1/1		0s	33ms/step
1/1		0s	33ms/step
1/1		0s	46ms/step
1/1		0s	36ms/step
1/1		0s	32ms/step
1/1		0s	32ms/step
1/1		0s	32ms/step
1/1		0s	35ms/step
1/1		0s	34ms/step
1/1		0s	37ms/step
1/1		0s	34ms/step
1/1		0s	33ms/step
1/1		0s	32ms/step
1/1		0s	35ms/step
1/1		0s	32ms/step
1/1		0s	33ms/step
1/1		0s	40ms/step
1/1		0s	33ms/step
1/1		0s	32ms/step
1/1		0s	33ms/step
1/1		0s	35ms/step
1/1		0s	34ms/step
1/1		0s	34ms/step
1/1		0s	34ms/step
1/1		0s	32ms/step
1/1		0s	35ms/step
1/1		0s	37ms/step
1/1		0s	38ms/step
1/1		0s	32ms/step
1/1		0s	36ms/step
1/1		0s	45ms/step
1/1		0s	36ms/step
1/1		0s	36ms/step
1/1		0s	36ms/step
1/1		0s	48ms/step
1/1		0s	48ms/step

1/1	_____	0s	41ms/step
1/1	_____	0s	54ms/step
1/1	_____	0s	39ms/step
1/1	_____	0s	50ms/step
1/1	_____	0s	63ms/step
1/1	_____	0s	44ms/step
1/1	_____	0s	47ms/step
1/1	_____	0s	48ms/step
1/1	_____	0s	45ms/step
1/1	_____	0s	48ms/step
1/1	_____	0s	52ms/step
1/1	_____	0s	57ms/step
1/1	_____	0s	56ms/step
1/1	_____	0s	45ms/step
1/1	_____	0s	53ms/step
1/1	_____	0s	48ms/step
1/1	_____	0s	37ms/step
1/1	_____	0s	52ms/step
1/1	_____	0s	47ms/step
1/1	_____	0s	48ms/step
1/1	_____	0s	54ms/step
1/1	_____	0s	55ms/step
1/1	_____	0s	55ms/step
1/1	_____	0s	58ms/step
1/1	_____	0s	51ms/step
1/1	_____	0s	47ms/step
1/1	_____	0s	51ms/step
1/1	_____	0s	36ms/step
1/1	_____	0s	32ms/step
1/1	_____	0s	32ms/step
1/1	_____	0s	31ms/step
1/1	_____	0s	35ms/step
1/1	_____	0s	38ms/step
1/1	_____	0s	36ms/step
1/1	_____	0s	34ms/step
1/1	_____	0s	36ms/step
1/1	_____	0s	41ms/step
1/1	_____	0s	36ms/step
1/1	_____	0s	35ms/step
1/1	_____	0s	36ms/step
1/1	_____	0s	32ms/step
1/1	_____	0s	36ms/step
1/1	_____	0s	31ms/step
1/1	_____	0s	35ms/step
1/1	_____	0s	34ms/step
1/1	_____	0s	38ms/step
1/1	_____	0s	33ms/step
1/1	_____	0s	39ms/step
1/1	_____	0s	33ms/step
1/1	_____	0s	35ms/step
1/1	_____	0s	37ms/step
1/1	_____	0s	35ms/step
1/1	_____	0s	33ms/step
1/1	_____	0s	33ms/step
1/1	_____	0s	35ms/step
1/1	_____	0s	36ms/step

1/1		0s	33ms/step
1/1		0s	35ms/step
1/1		0s	32ms/step
1/1		0s	36ms/step
1/1		0s	34ms/step
1/1		0s	34ms/step
1/1		0s	34ms/step
1/1		0s	41ms/step
1/1		0s	35ms/step
1/1		0s	34ms/step
1/1		0s	33ms/step
1/1		0s	37ms/step
1/1		0s	34ms/step
1/1		0s	37ms/step
1/1		0s	34ms/step
1/1		0s	36ms/step
1/1		0s	32ms/step
1/1		0s	34ms/step
1/1		0s	34ms/step
1/1		0s	34ms/step
1/1		0s	32ms/step
1/1		0s	33ms/step
1/1		0s	31ms/step
1/1		0s	37ms/step
1/1		0s	34ms/step
1/1		0s	34ms/step
1/1		0s	37ms/step
1/1		0s	34ms/step
1/1		0s	33ms/step
1/1		0s	36ms/step
1/1		0s	33ms/step
1/1		0s	37ms/step
1/1		0s	32ms/step
1/1		0s	33ms/step
1/1		0s	32ms/step
1/1		0s	37ms/step
1/1		0s	33ms/step
1/1		0s	36ms/step
1/1		0s	33ms/step
1/1		0s	36ms/step
1/1		0s	34ms/step
1/1		0s	34ms/step
1/1		0s	37ms/step
1/1		0s	38ms/step
1/1		0s	32ms/step
1/1		0s	33ms/step
1/1		0s	32ms/step
1/1		0s	36ms/step
1/1		0s	33ms/step
1/1		0s	42ms/step
1/1		0s	36ms/step
1/1		0s	36ms/step
1/1		0s	33ms/step
1/1		0s	37ms/step
1/1		0s	35ms/step
1/1		0s	35ms/step

1/1  0s 33ms/step
1/1  0s 35ms/step
1/1  0s 33ms/step
1/1  0s 38ms/step
1/1  0s 34ms/step
1/1  0s 37ms/step
1/1  0s 35ms/step
1/1  0s 50ms/step
1/1  0s 35ms/step

Accuracy 99.33

Checking 20 way accuracy....

1/1  1s 965ms/step
1/1  0s 33ms/step
1/1  0s 32ms/step
1/1  0s 34ms/step
1/1  0s 35ms/step
1/1  0s 31ms/step
1/1  0s 35ms/step
1/1  0s 33ms/step
1/1  0s 36ms/step
1/1  0s 35ms/step
1/1  0s 34ms/step
1/1  0s 34ms/step
1/1  0s 35ms/step
1/1  0s 33ms/step
1/1  0s 43ms/step
1/1  0s 31ms/step
1/1  0s 32ms/step
1/1  0s 32ms/step
1/1  0s 34ms/step
1/1  0s 33ms/step
1/1  0s 37ms/step
1/1  0s 35ms/step
1/1  0s 35ms/step
1/1  0s 35ms/step
1/1  0s 34ms/step
1/1  0s 37ms/step
1/1  0s 33ms/step
1/1  0s 42ms/step
1/1  0s 54ms/step
1/1  0s 49ms/step
1/1  0s 51ms/step
1/1  0s 57ms/step
1/1  0s 54ms/step
1/1  0s 54ms/step
1/1  0s 44ms/step
1/1  0s 50ms/step
1/1  0s 63ms/step
1/1  0s 65ms/step
1/1  0s 47ms/step
1/1  0s 51ms/step
1/1  0s 49ms/step
1/1  0s 45ms/step
1/1  0s 59ms/step
1/1  0s 43ms/step
1/1  0s 60ms/step

1/1	—————	0s	53ms/step
1/1	—————	0s	48ms/step
1/1	—————	0s	57ms/step
1/1	—————	0s	54ms/step
1/1	—————	0s	52ms/step
1/1	—————	0s	54ms/step
1/1	—————	0s	46ms/step
1/1	—————	0s	43ms/step
1/1	—————	0s	51ms/step
1/1	—————	0s	51ms/step
1/1	—————	0s	46ms/step
1/1	—————	0s	54ms/step
1/1	—————	0s	76ms/step
1/1	—————	0s	50ms/step
1/1	—————	0s	77ms/step
1/1	—————	0s	44ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	38ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	37ms/step
1/1	—————	0s	46ms/step
1/1	—————	0s	40ms/step
1/1	—————	0s	37ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	38ms/step
1/1	—————	0s	38ms/step
1/1	—————	0s	38ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	44ms/step
1/1	—————	0s	37ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	37ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	37ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	34ms/step

1/1	—————	0s	32ms/step
1/1	—————	0s	37ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	38ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	37ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	38ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	38ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	39ms/step
1/1	—————	0s	37ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	47ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	30ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	39ms/step
1/1	—————	0s	38ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	31ms/step
1/1	—————	0s	42ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	37ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	38ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	31ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	37ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	37ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	35ms/step

1/1	—————	0s	34ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	37ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	38ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	39ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	38ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	41ms/step
1/1	—————	0s	39ms/step
1/1	—————	0s	57ms/step
1/1	—————	0s	42ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	37ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	39ms/step
1/1	—————	0s	74ms/step
1/1	—————	0s	57ms/step
1/1	—————	0s	49ms/step
1/1	—————	0s	44ms/step
1/1	—————	0s	46ms/step
1/1	—————	0s	46ms/step
1/1	—————	0s	54ms/step
1/1	—————	0s	40ms/step
1/1	—————	0s	45ms/step
1/1	—————	0s	42ms/step
1/1	—————	0s	60ms/step
1/1	—————	0s	55ms/step
1/1	—————	0s	42ms/step
1/1	—————	0s	50ms/step
1/1	—————	0s	43ms/step
1/1	—————	0s	46ms/step
1/1	—————	0s	56ms/step
1/1	—————	0s	51ms/step
1/1	—————	0s	51ms/step
1/1	—————	0s	37ms/step
1/1	—————	0s	47ms/step
1/1	—————	0s	44ms/step
1/1	—————	0s	51ms/step
1/1	—————	0s	44ms/step
1/1	—————	0s	50ms/step

1/1	_____	0s	52ms/step
1/1	_____	0s	53ms/step
1/1	_____	0s	46ms/step
1/1	_____	0s	53ms/step
1/1	_____	0s	36ms/step
1/1	_____	0s	36ms/step
1/1	_____	0s	33ms/step
1/1	_____	0s	34ms/step
1/1	_____	0s	35ms/step
1/1	_____	0s	37ms/step
1/1	_____	0s	36ms/step
1/1	_____	0s	38ms/step
1/1	_____	0s	36ms/step
1/1	_____	0s	35ms/step
1/1	_____	0s	35ms/step
1/1	_____	0s	36ms/step
1/1	_____	0s	33ms/step
1/1	_____	0s	36ms/step
1/1	_____	0s	34ms/step
1/1	_____	0s	39ms/step
1/1	_____	0s	32ms/step
1/1	_____	0s	36ms/step
1/1	_____	0s	36ms/step
1/1	_____	0s	41ms/step
1/1	_____	0s	34ms/step
1/1	_____	0s	39ms/step
1/1	_____	0s	34ms/step
1/1	_____	0s	38ms/step
1/1	_____	0s	33ms/step
1/1	_____	0s	34ms/step
1/1	_____	0s	32ms/step
1/1	_____	0s	35ms/step
1/1	_____	0s	32ms/step
1/1	_____	0s	36ms/step
1/1	_____	0s	34ms/step
1/1	_____	0s	37ms/step
1/1	_____	0s	33ms/step
1/1	_____	0s	36ms/step
1/1	_____	0s	37ms/step
1/1	_____	0s	35ms/step
1/1	_____	0s	33ms/step
1/1	_____	0s	39ms/step
1/1	_____	0s	33ms/step
1/1	_____	0s	38ms/step
1/1	_____	0s	40ms/step
1/1	_____	0s	35ms/step
1/1	_____	0s	34ms/step
1/1	_____	0s	38ms/step
1/1	_____	0s	37ms/step
1/1	_____	0s	36ms/step
1/1	_____	0s	35ms/step
1/1	_____	0s	38ms/step
1/1	_____	0s	43ms/step
1/1	_____	0s	38ms/step
1/1	_____	0s	34ms/step
1/1	_____	0s	34ms/step

1/1	=====	0s	32ms/step
1/1	=====	0s	40ms/step
1/1	=====	0s	37ms/step
1/1	=====	0s	36ms/step
1/1	=====	0s	36ms/step
1/1	=====	0s	36ms/step
1/1	=====	0s	34ms/step
1/1	=====	0s	36ms/step
1/1	=====	0s	35ms/step
1/1	=====	0s	42ms/step
1/1	=====	0s	34ms/step
1/1	=====	0s	37ms/step
1/1	=====	0s	36ms/step
1/1	=====	0s	41ms/step
1/1	=====	0s	35ms/step
1/1	=====	0s	40ms/step
1/1	=====	0s	37ms/step
1/1	=====	0s	39ms/step
1/1	=====	0s	33ms/step
1/1	=====	0s	36ms/step
1/1	=====	0s	33ms/step
1/1	=====	0s	35ms/step
1/1	=====	0s	32ms/step
1/1	=====	0s	38ms/step
1/1	=====	0s	34ms/step
1/1	=====	0s	37ms/step
1/1	=====	0s	33ms/step
1/1	=====	0s	38ms/step
1/1	=====	0s	34ms/step
1/1	=====	0s	36ms/step
1/1	=====	0s	39ms/step

Accuracy 98.67

Checking 25 way accuracy....

1/1	=====	1s	994ms/step
1/1	=====	0s	33ms/step
1/1	=====	0s	37ms/step
1/1	=====	0s	34ms/step
1/1	=====	0s	34ms/step
1/1	=====	0s	47ms/step
1/1	=====	0s	36ms/step
1/1	=====	0s	36ms/step
1/1	=====	0s	38ms/step
1/1	=====	0s	35ms/step
1/1	=====	0s	36ms/step
1/1	=====	0s	35ms/step
1/1	=====	0s	36ms/step
1/1	=====	0s	35ms/step
1/1	=====	0s	40ms/step
1/1	=====	0s	33ms/step
1/1	=====	0s	36ms/step
1/1	=====	0s	41ms/step
1/1	=====	0s	37ms/step
1/1	=====	0s	36ms/step
1/1	=====	0s	40ms/step
1/1	=====	0s	36ms/step
1/1	=====	0s	44ms/step

1/1	—————	0s	36ms/step
1/1	—————	0s	40ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	47ms/step
1/1	—————	0s	69ms/step
1/1	—————	0s	73ms/step
1/1	—————	0s	50ms/step
1/1	—————	0s	50ms/step
1/1	—————	0s	44ms/step
1/1	—————	0s	54ms/step
1/1	—————	0s	45ms/step
1/1	—————	0s	54ms/step
1/1	—————	0s	47ms/step
1/1	—————	0s	54ms/step
1/1	—————	0s	50ms/step
1/1	—————	0s	49ms/step
1/1	—————	0s	55ms/step
1/1	—————	0s	46ms/step
1/1	—————	0s	41ms/step
1/1	—————	0s	44ms/step
1/1	—————	0s	43ms/step
1/1	—————	0s	53ms/step
1/1	—————	0s	53ms/step
1/1	—————	0s	54ms/step
1/1	—————	0s	55ms/step
1/1	—————	0s	57ms/step
1/1	—————	0s	51ms/step
1/1	—————	0s	53ms/step
1/1	—————	0s	41ms/step
1/1	—————	0s	64ms/step
1/1	—————	0s	54ms/step
1/1	—————	0s	56ms/step
1/1	—————	0s	49ms/step
1/1	—————	0s	66ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	37ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	37ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	31ms/step
1/1	—————	0s	39ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	38ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	38ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	38ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	40ms/step

1/1	—————	0s	33ms/step
1/1	—————	0s	38ms/step
1/1	—————	0s	37ms/step
1/1	—————	0s	49ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	38ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	37ms/step
1/1	—————	0s	42ms/step
1/1	—————	0s	41ms/step
1/1	—————	0s	38ms/step
1/1	—————	0s	41ms/step
1/1	—————	0s	37ms/step
1/1	—————	0s	37ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	39ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	37ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	37ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	38ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	38ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	40ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	38ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	39ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	38ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	37ms/step
1/1	—————	0s	38ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	37ms/step
1/1	—————	0s	38ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	37ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	39ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	38ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	58ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	42ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	37ms/step

1/1	_____	0s	35ms/step
1/1	_____	0s	42ms/step
1/1	_____	0s	34ms/step
1/1	_____	0s	40ms/step
1/1	_____	0s	37ms/step
1/1	_____	0s	38ms/step
1/1	_____	0s	36ms/step
1/1	_____	0s	39ms/step
1/1	_____	0s	36ms/step
1/1	_____	0s	37ms/step
1/1	_____	0s	34ms/step
1/1	_____	0s	36ms/step
1/1	_____	0s	41ms/step
1/1	_____	0s	35ms/step
1/1	_____	0s	33ms/step
1/1	_____	0s	38ms/step
1/1	_____	0s	35ms/step
1/1	_____	0s	37ms/step
1/1	_____	0s	37ms/step
1/1	_____	0s	37ms/step
1/1	_____	0s	36ms/step
1/1	_____	0s	39ms/step
1/1	_____	0s	33ms/step
1/1	_____	0s	35ms/step
1/1	_____	0s	32ms/step
1/1	_____	0s	38ms/step
1/1	_____	0s	32ms/step
1/1	_____	0s	39ms/step
1/1	_____	0s	39ms/step
1/1	_____	0s	37ms/step
1/1	_____	0s	35ms/step
1/1	_____	0s	39ms/step
1/1	_____	0s	35ms/step
1/1	_____	0s	36ms/step
1/1	_____	0s	36ms/step
1/1	_____	0s	37ms/step
1/1	_____	0s	35ms/step
1/1	_____	0s	36ms/step
1/1	_____	0s	35ms/step
1/1	_____	0s	37ms/step
1/1	_____	0s	60ms/step
1/1	_____	0s	53ms/step
1/1	_____	0s	53ms/step
1/1	_____	0s	67ms/step
1/1	_____	0s	55ms/step
1/1	_____	0s	56ms/step
1/1	_____	0s	44ms/step
1/1	_____	0s	48ms/step
1/1	_____	0s	40ms/step
1/1	_____	0s	61ms/step
1/1	_____	0s	53ms/step
1/1	_____	0s	54ms/step
1/1	_____	0s	40ms/step
1/1	_____	0s	46ms/step
1/1	_____	0s	59ms/step
1/1	_____	0s	47ms/step

1/1	—————	0s	44ms/step
1/1	—————	0s	75ms/step
1/1	—————	0s	54ms/step
1/1	—————	0s	59ms/step
1/1	—————	0s	46ms/step
1/1	—————	0s	42ms/step
1/1	—————	0s	49ms/step
1/1	—————	0s	64ms/step
1/1	—————	0s	53ms/step
1/1	—————	0s	49ms/step
1/1	—————	0s	51ms/step
1/1	—————	0s	58ms/step
1/1	—————	0s	51ms/step
1/1	—————	0s	61ms/step
1/1	—————	0s	45ms/step
1/1	—————	0s	47ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	40ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	38ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	41ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	41ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	38ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	38ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	41ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	37ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	40ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	38ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	37ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	38ms/step
1/1	—————	0s	37ms/step
1/1	—————	0s	45ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	39ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	37ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	41ms/step

1/1	0s	39ms/step
1/1	0s	39ms/step
1/1	0s	35ms/step
1/1	0s	39ms/step
1/1	0s	39ms/step
1/1	0s	38ms/step
1/1	0s	34ms/step
1/1	0s	37ms/step
1/1	0s	37ms/step
1/1	0s	40ms/step
1/1	0s	34ms/step
1/1	0s	39ms/step
1/1	0s	35ms/step
1/1	0s	38ms/step
1/1	0s	37ms/step
1/1	0s	38ms/step
1/1	0s	34ms/step
1/1	0s	38ms/step
1/1	0s	36ms/step
1/1	0s	35ms/step
1/1	0s	32ms/step
1/1	0s	35ms/step
1/1	0s	34ms/step
1/1	0s	37ms/step
1/1	0s	38ms/step
1/1	0s	38ms/step
1/1	0s	36ms/step
1/1	0s	41ms/step
1/1	0s	37ms/step
1/1	0s	45ms/step
1/1	0s	40ms/step
1/1	0s	36ms/step
1/1	0s	33ms/step
1/1	0s	37ms/step
1/1	0s	33ms/step
1/1	0s	38ms/step
1/1	0s	37ms/step
1/1	0s	37ms/step
1/1	0s	36ms/step
1/1	0s	53ms/step
1/1	0s	34ms/step
1/1	0s	39ms/step
1/1	0s	36ms/step
1/1	0s	38ms/step
1/1	0s	36ms/step
1/1	0s	40ms/step
1/1	0s	36ms/step
1/1	0s	40ms/step
1/1	0s	35ms/step
1/1	0s	39ms/step
1/1	0s	35ms/step
1/1	0s	41ms/step
1/1	0s	38ms/step

Accuracy 100.00

Share your Observations

Write your answer here:

Looking at the Loss Curve, following are my observation:

- The loss curve decreases very quickly during the first two epochs and then continues to fall steadily across the remaining epochs. This shows that the Siamese network is learning the embedding space correctly and efficiently.
- By the final epochs, the loss value becomes extremely small, which means the model is successfully reducing the distance between similar image pairs and increasing the distance between dissimilar pairs.
- The smooth downward trend without any sudden jumps also indicates stable training with no signs of overfitting or training instability.

Looking at the Accuracy results, following are my observations:

- The Accuracy was 100% for N-Way = 5
- As the N-Way, increases the accuracy slightly fluctuates (slight variations).
- This is expected, since when the N-Way was lower, model has to make a choice between 5 or 10 faces, while it increases model now needs to make choice from a larger set, making probability of mistakes more higher. However it becomes 100% again at N-way = 25.
- The model still performs quite well even as the difficulty increases.
- The result match what we expect from a Triplet_loss_Trained Siamese Network and it learns good separation between same and different faces.
- Because in above code we computed the validation accuracy (train_data = False), the result shows that model is generalizing well and not just memorizing the training set.

In simple words: Although the accuracy naturally drops with more classes to compare, the model still maintains good performance, which suggests that the embedding it learns are reliable for face similarity tasks.

- The code sets the variable test_sample to 150, nway_min to 5, and nway_max to 25. It also initializes two empty lists, test_acc and nway_list.
- The code enters a for loop that iterates through a range of values from nway_min to nway_max with a step of 5.
- Within the for loop, it prints "Checking %d way accuracy..." % nway, where nway is the current value in the range.
- It also initializes a variable correct_count to 0.
- The code then enters another for loop that iterates test_sample number of times.
- Within this inner for loop, the code calls the function get_triplet_batch from the preprocess_data module, passing in the current value of nway and the value False for train_data. This function returns three sets of data: nway_anchor, nway_positive, and _ (which is not used in the code).

- The code then uses the embedding_model to predict the embedding vector for the nway_anchor data. This value is stored in the variable nway_anchor_embedding.
- The code then randomly selects an index from 0 to nway-1 and uses this index to select a sample from the nway_positive data. It uses the embedding_model to predict the embedding vector for this sample, and stores the result in the variable sample_embedding.
- The code calculates the distance between the nway_anchor_embedding and sample_embedding using the TensorFlow functions sqrt, reduce_sum, and pow. It stores the result in the variable distance.
- The code checks if the index of the minimum value in the distance variable is equal to the randomly selected sample_index. If it is, it increments the correct_count variable by 1.
- After the inner for loop is finished, the code appends the current value of nway to the nway_list and calculates the accuracy as correct_count divided by test_sample times 100. It then appends this value to the test_acc list.
- The code then prints "Accuracy %.2f" % acc, where acc is the current accuracy calculated in the previous step.
- The for loop continues until it has iterated through all the values in the range(nway_min, nway_max+1,5).
- After the for loop is finished, the code has stored the accuracy for each value of nway in the range in the test_acc list and the corresponding value of nway in the nway_list.

```
In [23]: test_sample = 150
nway_min = 5
nway_max = 25
train_acc = []
nway_list = []
for nway in range(nway_min, nway_max+1,5):
    print("Checking %d way accuracy...." % nway)
    correct_count = 0
    for _ in range(test_sample):
        nway_anchor, nway_positive, _ = preprocess_data.get_triplet_batch(nway, train_d
        nway_anchor_embedding = emb_model.predict(nway_anchor)
        sample_index = random.randint(0,nway-1)
        sample_embedding = emb_model.predict(np.expand_dims(nway_positive[sample_index]
        distance = tf.sqrt(tf.reduce_sum(tf.pow(nway_anchor_embedding - sample_embeddin
        if np.argmin(distance) == sample_index:
            correct_count+=1
    nway_list.append(nway)
    acc = (correct_count/test_sample)*100.
    train_acc.append(acc)
    print("Accuracy %.2f" % acc)
```

Checking 5 way accuracy....

1/1	=====	0s	33ms/step
1/1	=====	0s	29ms/step
1/1	=====	0s	31ms/step
1/1	=====	0s	34ms/step
1/1	=====	0s	49ms/step
1/1	=====	0s	44ms/step
1/1	=====	0s	46ms/step
1/1	=====	0s	41ms/step
1/1	=====	0s	43ms/step
1/1	=====	0s	43ms/step
1/1	=====	0s	45ms/step
1/1	=====	0s	44ms/step
1/1	=====	0s	46ms/step
1/1	=====	0s	41ms/step
1/1	=====	0s	43ms/step
1/1	=====	0s	41ms/step
1/1	=====	0s	40ms/step
1/1	=====	0s	50ms/step
1/1	=====	0s	45ms/step
1/1	=====	0s	40ms/step
1/1	=====	0s	41ms/step
1/1	=====	0s	38ms/step
1/1	=====	0s	39ms/step
1/1	=====	0s	51ms/step
1/1	=====	0s	55ms/step
1/1	=====	0s	48ms/step
1/1	=====	0s	47ms/step
1/1	=====	0s	43ms/step
1/1	=====	0s	49ms/step
1/1	=====	0s	45ms/step
1/1	=====	0s	55ms/step
1/1	=====	0s	53ms/step
1/1	=====	0s	50ms/step
1/1	=====	0s	40ms/step
1/1	=====	0s	32ms/step
1/1	=====	0s	29ms/step
1/1	=====	0s	29ms/step
1/1	=====	0s	28ms/step
1/1	=====	0s	29ms/step
1/1	=====	0s	30ms/step
1/1	=====	0s	28ms/step
1/1	=====	0s	27ms/step
1/1	=====	0s	30ms/step
1/1	=====	0s	30ms/step
1/1	=====	0s	32ms/step
1/1	=====	0s	29ms/step
1/1	=====	0s	30ms/step
1/1	=====	0s	29ms/step
1/1	=====	0s	29ms/step
1/1	=====	0s	28ms/step
1/1	=====	0s	29ms/step
1/1	=====	0s	28ms/step
1/1	=====	0s	28ms/step
1/1	=====	0s	29ms/step
1/1	=====	0s	30ms/step

1/1		0s	30ms/step
1/1		0s	32ms/step
1/1		0s	30ms/step
1/1		0s	30ms/step
1/1		0s	29ms/step
1/1		0s	30ms/step
1/1		0s	30ms/step
1/1		0s	29ms/step
1/1		0s	29ms/step
1/1		0s	28ms/step
1/1		0s	29ms/step
1/1		0s	40ms/step
1/1		0s	29ms/step
1/1		0s	29ms/step
1/1		0s	30ms/step
1/1		0s	30ms/step
1/1		0s	30ms/step
1/1		0s	29ms/step
1/1		0s	28ms/step
1/1		0s	28ms/step
1/1		0s	28ms/step
1/1		0s	29ms/step
1/1		0s	28ms/step
1/1		0s	32ms/step
1/1		0s	32ms/step
1/1		0s	32ms/step
1/1		0s	29ms/step
1/1		0s	31ms/step
1/1		0s	37ms/step
1/1		0s	29ms/step
1/1		0s	28ms/step
1/1		0s	31ms/step
1/1		0s	31ms/step
1/1		0s	31ms/step
1/1		0s	29ms/step
1/1		0s	33ms/step
1/1		0s	30ms/step
1/1		0s	33ms/step
1/1		0s	30ms/step
1/1		0s	32ms/step
1/1		0s	31ms/step
1/1		0s	30ms/step
1/1		0s	29ms/step
1/1		0s	30ms/step
1/1		0s	41ms/step
1/1		0s	30ms/step
1/1		0s	28ms/step
1/1		0s	33ms/step
1/1		0s	31ms/step
1/1		0s	31ms/step
1/1		0s	31ms/step
1/1		0s	32ms/step
1/1		0s	29ms/step
1/1		0s	31ms/step
1/1		0s	30ms/step
1/1		0s	31ms/step

1/1		0s	29ms/step
1/1		0s	29ms/step
1/1		0s	29ms/step
1/1		0s	31ms/step
1/1		0s	42ms/step
1/1		0s	30ms/step
1/1		0s	29ms/step
1/1		0s	32ms/step
1/1		0s	33ms/step
1/1		0s	30ms/step
1/1		0s	28ms/step
1/1		0s	29ms/step
1/1		0s	33ms/step
1/1		0s	30ms/step
1/1		0s	29ms/step
1/1		0s	31ms/step
1/1		0s	31ms/step
1/1		0s	30ms/step
1/1		0s	29ms/step
1/1		0s	30ms/step
1/1		0s	38ms/step
1/1		0s	29ms/step
1/1		0s	28ms/step
1/1		0s	31ms/step
1/1		0s	29ms/step
1/1		0s	30ms/step
1/1		0s	29ms/step
1/1		0s	32ms/step
1/1		0s	30ms/step
1/1		0s	32ms/step
1/1		0s	30ms/step
1/1		0s	30ms/step
1/1		0s	29ms/step
1/1		0s	31ms/step
1/1		0s	30ms/step
1/1		0s	30ms/step
1/1		0s	43ms/step
1/1		0s	33ms/step
1/1		0s	34ms/step
1/1		0s	30ms/step
1/1		0s	33ms/step
1/1		0s	31ms/step
1/1		0s	30ms/step
1/1		0s	32ms/step
1/1		0s	33ms/step
1/1		0s	30ms/step
1/1		0s	29ms/step
1/1		0s	46ms/step
1/1		0s	33ms/step
1/1		0s	32ms/step
1/1		0s	30ms/step
1/1		0s	40ms/step
1/1		0s	33ms/step
1/1		0s	33ms/step
1/1		0s	31ms/step
1/1		0s	31ms/step

1/1	—————	0s	31ms/step
1/1	—————	0s	30ms/step
1/1	—————	0s	31ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	30ms/step
1/1	—————	0s	30ms/step
1/1	—————	0s	28ms/step
1/1	—————	0s	30ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	30ms/step
1/1	—————	0s	30ms/step
1/1	—————	0s	39ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	31ms/step
1/1	—————	0s	29ms/step
1/1	—————	0s	31ms/step
1/1	—————	0s	31ms/step
1/1	—————	0s	29ms/step
1/1	—————	0s	30ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	54ms/step
1/1	—————	0s	46ms/step
1/1	—————	0s	53ms/step
1/1	—————	0s	51ms/step
1/1	—————	0s	45ms/step
1/1	—————	0s	47ms/step
1/1	—————	0s	45ms/step
1/1	—————	0s	46ms/step
1/1	—————	0s	41ms/step
1/1	—————	0s	38ms/step
1/1	—————	0s	49ms/step
1/1	—————	0s	44ms/step
1/1	—————	0s	42ms/step
1/1	—————	0s	45ms/step
1/1	—————	0s	49ms/step
1/1	—————	0s	42ms/step
1/1	—————	0s	45ms/step
1/1	—————	0s	42ms/step
1/1	—————	0s	43ms/step
1/1	—————	0s	43ms/step
1/1	—————	0s	41ms/step
1/1	—————	0s	42ms/step
1/1	—————	0s	53ms/step
1/1	—————	0s	51ms/step
1/1	—————	0s	57ms/step
1/1	—————	0s	41ms/step
1/1	—————	0s	53ms/step
1/1	—————	0s	46ms/step
1/1	—————	0s	54ms/step
1/1	—————	0s	51ms/step
1/1	—————	0s	50ms/step
1/1	—————	0s	29ms/step
1/1	—————	0s	31ms/step
1/1	—————	0s	28ms/step
1/1	—————	0s	31ms/step

1/1		0s	35ms/step
1/1		0s	37ms/step
1/1		0s	36ms/step
1/1		0s	36ms/step
1/1		0s	30ms/step
1/1		0s	33ms/step
1/1		0s	31ms/step
1/1		0s	31ms/step
1/1		0s	28ms/step
1/1		0s	31ms/step
1/1		0s	29ms/step
1/1		0s	31ms/step
1/1		0s	30ms/step
1/1		0s	33ms/step
1/1		0s	30ms/step
1/1		0s	31ms/step
1/1		0s	29ms/step
1/1		0s	32ms/step
1/1		0s	29ms/step
1/1		0s	29ms/step
1/1		0s	28ms/step
1/1		0s	31ms/step
1/1		0s	28ms/step
1/1		0s	32ms/step
1/1		0s	31ms/step
1/1		0s	33ms/step
1/1		0s	32ms/step
1/1		0s	31ms/step
1/1		0s	29ms/step
1/1		0s	31ms/step
1/1		0s	28ms/step
1/1		0s	30ms/step
1/1		0s	28ms/step
1/1		0s	40ms/step
1/1		0s	28ms/step
1/1		0s	32ms/step
1/1		0s	31ms/step
1/1		0s	32ms/step
1/1		0s	31ms/step
1/1		0s	31ms/step
1/1		0s	29ms/step
1/1		0s	33ms/step
1/1		0s	29ms/step
1/1		0s	30ms/step
1/1		0s	30ms/step
1/1		0s	30ms/step
1/1		0s	29ms/step
1/1		0s	31ms/step
1/1		0s	30ms/step
1/1		0s	60ms/step
1/1		0s	31ms/step
1/1		0s	35ms/step
1/1		0s	30ms/step
1/1		0s	34ms/step
1/1		0s	30ms/step
1/1		0s	29ms/step

```
1/1 _____ 0s 31ms/step
1/1 _____ 0s 31ms/step
1/1 _____ 0s 29ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 31ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 31ms/step
1/1 _____ 0s 31ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 31ms/step
1/1 _____ 0s 44ms/step
1/1 _____ 0s 29ms/step
1/1 _____ 0s 31ms/step
1/1 _____ 0s 29ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 31ms/step
1/1 _____ 0s 31ms/step
1/1 _____ 0s 30ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 31ms/step
```

Accuracy 99.33

Checking 10 way accuracy....

```
1/1 _____ 0s 32ms/step
1/1 _____ 0s 31ms/step
1/1 _____ 0s 31ms/step
1/1 _____ 0s 30ms/step
1/1 _____ 0s 30ms/step
1/1 _____ 0s 30ms/step
1/1 _____ 0s 42ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 30ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 31ms/step
1/1 _____ 0s 31ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 29ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 35ms/step
1/1 _____ 0s 35ms/step
1/1 _____ 0s 31ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 29ms/step
1/1 _____ 0s 30ms/step
1/1 _____ 0s 29ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 28ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 30ms/step
1/1 _____ 0s 32ms/step
```

1/1	—————	0s	31ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	30ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	29ms/step
1/1	—————	0s	31ms/step
1/1	—————	0s	29ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	31ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	31ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	30ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	30ms/step
1/1	—————	0s	42ms/step
1/1	—————	0s	30ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	31ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	31ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	51ms/step
1/1	—————	0s	52ms/step
1/1	—————	0s	45ms/step
1/1	—————	0s	41ms/step
1/1	—————	0s	49ms/step
1/1	—————	0s	47ms/step
1/1	—————	0s	51ms/step
1/1	—————	0s	46ms/step
1/1	—————	0s	54ms/step
1/1	—————	0s	47ms/step
1/1	—————	0s	50ms/step
1/1	—————	0s	46ms/step
1/1	—————	0s	51ms/step
1/1	—————	0s	47ms/step
1/1	—————	0s	46ms/step
1/1	—————	0s	45ms/step
1/1	—————	0s	49ms/step
1/1	—————	0s	48ms/step
1/1	—————	0s	45ms/step
1/1	—————	0s	39ms/step
1/1	—————	0s	50ms/step

1/1	—————	0s	42ms/step
1/1	—————	0s	50ms/step
1/1	—————	0s	47ms/step
1/1	—————	0s	55ms/step
1/1	—————	0s	57ms/step
1/1	—————	0s	50ms/step
1/1	—————	0s	54ms/step
1/1	—————	0s	37ms/step
1/1	—————	0s	29ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	29ms/step
1/1	—————	0s	30ms/step
1/1	—————	0s	29ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	38ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	31ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	30ms/step
1/1	—————	0s	31ms/step
1/1	—————	0s	30ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	30ms/step
1/1	—————	0s	30ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	31ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	30ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	29ms/step
1/1	—————	0s	31ms/step
1/1	—————	0s	37ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	31ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	31ms/step
1/1	—————	0s	30ms/step
1/1	—————	0s	31ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	44ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	37ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	33ms/step

1/1		0s	32ms/step
1/1		0s	32ms/step
1/1		0s	32ms/step
1/1		0s	32ms/step
1/1		0s	32ms/step
1/1		0s	32ms/step
1/1		0s	31ms/step
1/1		0s	34ms/step
1/1		0s	34ms/step
1/1		0s	32ms/step
1/1		0s	32ms/step
1/1		0s	33ms/step
1/1		0s	31ms/step
1/1		0s	34ms/step
1/1		0s	32ms/step
1/1		0s	32ms/step
1/1		0s	31ms/step
1/1		0s	33ms/step
1/1		0s	32ms/step
1/1		0s	36ms/step
1/1		0s	32ms/step
1/1		0s	33ms/step
1/1		0s	32ms/step
1/1		0s	35ms/step
1/1		0s	32ms/step
1/1		0s	37ms/step
1/1		0s	31ms/step
1/1		0s	31ms/step
1/1		0s	30ms/step
1/1		0s	34ms/step
1/1		0s	31ms/step
1/1		0s	33ms/step
1/1		0s	32ms/step
1/1		0s	32ms/step
1/1		0s	33ms/step
1/1		0s	31ms/step
1/1		0s	32ms/step
1/1		0s	36ms/step
1/1		0s	34ms/step
1/1		0s	41ms/step
1/1		0s	30ms/step
1/1		0s	33ms/step
1/1		0s	31ms/step
1/1		0s	34ms/step
1/1		0s	31ms/step
1/1		0s	35ms/step
1/1		0s	32ms/step
1/1		0s	33ms/step
1/1		0s	30ms/step
1/1		0s	31ms/step
1/1		0s	30ms/step
1/1		0s	32ms/step
1/1		0s	30ms/step
1/1		0s	34ms/step
1/1		0s	34ms/step
1/1		0s	33ms/step

1/1	—————	0s	33ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	31ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	31ms/step
1/1	—————	0s	31ms/step
1/1	—————	0s	29ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	30ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	31ms/step
1/1	—————	0s	31ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	31ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	40ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	30ms/step
1/1	—————	0s	31ms/step
1/1	—————	0s	31ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	50ms/step
1/1	—————	0s	58ms/step
1/1	—————	0s	48ms/step
1/1	—————	0s	49ms/step
1/1	—————	0s	40ms/step
1/1	—————	0s	49ms/step
1/1	—————	0s	46ms/step
1/1	—————	0s	53ms/step
1/1	—————	0s	47ms/step
1/1	—————	0s	47ms/step
1/1	—————	0s	42ms/step
1/1	—————	0s	45ms/step
1/1	—————	0s	68ms/step
1/1	—————	0s	52ms/step
1/1	—————	0s	52ms/step
1/1	—————	0s	53ms/step
1/1	—————	0s	47ms/step
1/1	—————	0s	41ms/step

1/1	_____	0s	42ms/step
1/1	_____	0s	49ms/step
1/1	_____	0s	53ms/step
1/1	_____	0s	48ms/step
1/1	_____	0s	59ms/step
1/1	_____	0s	46ms/step
1/1	_____	0s	48ms/step
1/1	_____	0s	57ms/step
1/1	_____	0s	46ms/step
1/1	_____	0s	52ms/step
1/1	_____	0s	44ms/step
1/1	_____	0s	50ms/step
1/1	_____	0s	32ms/step
1/1	_____	0s	33ms/step
1/1	_____	0s	34ms/step
1/1	_____	0s	42ms/step
1/1	_____	0s	33ms/step
1/1	_____	0s	33ms/step
1/1	_____	0s	31ms/step
1/1	_____	0s	34ms/step
1/1	_____	0s	31ms/step
1/1	_____	0s	35ms/step
1/1	_____	0s	29ms/step
1/1	_____	0s	30ms/step
1/1	_____	0s	29ms/step
1/1	_____	0s	36ms/step
1/1	_____	0s	33ms/step
1/1	_____	0s	35ms/step
1/1	_____	0s	32ms/step
1/1	_____	0s	34ms/step
1/1	_____	0s	33ms/step
1/1	_____	0s	34ms/step
1/1	_____	0s	30ms/step
1/1	_____	0s	32ms/step
1/1	_____	0s	32ms/step
1/1	_____	0s	34ms/step
1/1	_____	0s	30ms/step
1/1	_____	0s	35ms/step
1/1	_____	0s	33ms/step
1/1	_____	0s	36ms/step
1/1	_____	0s	31ms/step
1/1	_____	0s	33ms/step
1/1	_____	0s	31ms/step

Accuracy 100.00

Checking 15 way accuracy....

1/1	_____	0s	35ms/step
1/1	_____	0s	30ms/step
1/1	_____	0s	33ms/step
1/1	_____	0s	32ms/step
1/1	_____	0s	34ms/step
1/1	_____	0s	31ms/step
1/1	_____	0s	34ms/step
1/1	_____	0s	33ms/step
1/1	_____	0s	36ms/step
1/1	_____	0s	33ms/step
1/1	_____	0s	35ms/step

1/1		0s	35ms/step
1/1		0s	35ms/step
1/1		0s	31ms/step
1/1		0s	32ms/step
1/1		0s	30ms/step
1/1		0s	44ms/step
1/1		0s	32ms/step
1/1		0s	34ms/step
1/1		0s	33ms/step
1/1		0s	35ms/step
1/1		0s	31ms/step
1/1		0s	34ms/step
1/1		0s	31ms/step
1/1		0s	34ms/step
1/1		0s	31ms/step
1/1		0s	32ms/step
1/1		0s	30ms/step
1/1		0s	32ms/step
1/1		0s	29ms/step
1/1		0s	33ms/step
1/1		0s	37ms/step
1/1		0s	34ms/step
1/1		0s	31ms/step
1/1		0s	35ms/step
1/1		0s	32ms/step
1/1		0s	38ms/step
1/1		0s	31ms/step
1/1		0s	34ms/step
1/1		0s	30ms/step
1/1		0s	33ms/step
1/1		0s	30ms/step
1/1		0s	33ms/step
1/1		0s	32ms/step
1/1		0s	35ms/step
1/1		0s	31ms/step
1/1		0s	37ms/step
1/1		0s	31ms/step
1/1		0s	34ms/step
1/1		0s	32ms/step
1/1		0s	33ms/step
1/1		0s	31ms/step
1/1		0s	33ms/step
1/1		0s	31ms/step
1/1		0s	34ms/step
1/1		0s	32ms/step
1/1		0s	35ms/step
1/1		0s	31ms/step
1/1		0s	34ms/step
1/1		0s	34ms/step
1/1		0s	39ms/step
1/1		0s	33ms/step
1/1		0s	37ms/step
1/1		0s	33ms/step
1/1		0s	33ms/step
1/1		0s	30ms/step
1/1		0s	35ms/step

1/1	—————	0s	32ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	39ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	31ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	29ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	37ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	31ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	31ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	31ms/step
1/1	—————	0s	38ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	31ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	30ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	48ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	31ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	51ms/step
1/1	—————	0s	48ms/step
1/1	—————	0s	55ms/step
1/1	—————	0s	40ms/step
1/1	—————	0s	55ms/step
1/1	—————	0s	67ms/step
1/1	—————	0s	56ms/step
1/1	—————	0s	48ms/step
1/1	—————	0s	52ms/step
1/1	—————	0s	50ms/step
1/1	—————	0s	48ms/step

1/1	—————	0s	40ms/step
1/1	—————	0s	42ms/step
1/1	—————	0s	41ms/step
1/1	—————	0s	53ms/step
1/1	—————	0s	53ms/step
1/1	—————	0s	51ms/step
1/1	—————	0s	52ms/step
1/1	—————	0s	41ms/step
1/1	—————	0s	50ms/step
1/1	—————	0s	44ms/step
1/1	—————	0s	41ms/step
1/1	—————	0s	47ms/step
1/1	—————	0s	50ms/step
1/1	—————	0s	52ms/step
1/1	—————	0s	58ms/step
1/1	—————	0s	47ms/step
1/1	—————	0s	49ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	30ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	40ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	31ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	37ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	42ms/step
1/1	—————	0s	37ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	35ms/step

1/1		0s	35ms/step
1/1		0s	37ms/step
1/1		0s	32ms/step
1/1		0s	37ms/step
1/1		0s	32ms/step
1/1		0s	35ms/step
1/1		0s	30ms/step
1/1		0s	35ms/step
1/1		0s	36ms/step
1/1		0s	35ms/step
1/1		0s	32ms/step
1/1		0s	35ms/step
1/1		0s	42ms/step
1/1		0s	35ms/step
1/1		0s	34ms/step
1/1		0s	35ms/step
1/1		0s	34ms/step
1/1		0s	32ms/step
1/1		0s	32ms/step
1/1		0s	35ms/step
1/1		0s	33ms/step
1/1		0s	35ms/step
1/1		0s	35ms/step
1/1		0s	36ms/step
1/1		0s	35ms/step
1/1		0s	35ms/step
1/1		0s	32ms/step
1/1		0s	47ms/step
1/1		0s	32ms/step
1/1		0s	31ms/step
1/1		0s	31ms/step
1/1		0s	35ms/step
1/1		0s	36ms/step
1/1		0s	41ms/step
1/1		0s	32ms/step
1/1		0s	38ms/step
1/1		0s	34ms/step
1/1		0s	33ms/step
1/1		0s	32ms/step
1/1		0s	36ms/step
1/1		0s	34ms/step
1/1		0s	34ms/step
1/1		0s	32ms/step
1/1		0s	34ms/step
1/1		0s	34ms/step
1/1		0s	34ms/step
1/1		0s	36ms/step
1/1		0s	34ms/step
1/1		0s	35ms/step
1/1		0s	32ms/step
1/1		0s	35ms/step
1/1		0s	32ms/step
1/1		0s	32ms/step
1/1		0s	30ms/step
1/1		0s	36ms/step

1/1	—————	0s	46ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	41ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	31ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	31ms/step
1/1	—————	0s	40ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	37ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	37ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	31ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	31ms/step
1/1	—————	0s	38ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	37ms/step
1/1	—————	0s	38ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	43ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	37ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	31ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	45ms/step
1/1	—————	0s	51ms/step
1/1	—————	0s	40ms/step
1/1	—————	0s	50ms/step
1/1	—————	0s	51ms/step
1/1	—————	0s	54ms/step
1/1	—————	0s	54ms/step
1/1	—————	0s	43ms/step
1/1	—————	0s	42ms/step
1/1	—————	0s	59ms/step
1/1	—————	0s	42ms/step
1/1	—————	0s	42ms/step

1/1  0s 38ms/step
1/1  0s 48ms/step
1/1  0s 46ms/step
1/1  0s 51ms/step
1/1  0s 50ms/step
1/1  0s 53ms/step
1/1  0s 68ms/step
1/1  0s 54ms/step
1/1  0s 51ms/step

Accuracy 99.33

Checking 20 way accuracy....

1/1  0s 49ms/step
1/1  0s 49ms/step
1/1  0s 51ms/step
1/1  0s 53ms/step
1/1  0s 43ms/step
1/1  0s 49ms/step
1/1  0s 47ms/step
1/1  0s 45ms/step
1/1  0s 53ms/step
1/1  0s 34ms/step
1/1  0s 36ms/step
1/1  0s 33ms/step
1/1  0s 36ms/step
1/1  0s 36ms/step
1/1  0s 34ms/step
1/1  0s 32ms/step
1/1  0s 35ms/step
1/1  0s 34ms/step
1/1  0s 37ms/step
1/1  0s 38ms/step
1/1  0s 36ms/step
1/1  0s 34ms/step
1/1  0s 36ms/step
1/1  0s 34ms/step
1/1  0s 37ms/step
1/1  0s 33ms/step
1/1  0s 35ms/step
1/1  0s 36ms/step
1/1  0s 37ms/step
1/1  0s 33ms/step
1/1  0s 40ms/step
1/1  0s 39ms/step
1/1  0s 43ms/step
1/1  0s 36ms/step
1/1  0s 48ms/step
1/1  0s 35ms/step
1/1  0s 35ms/step
1/1  0s 35ms/step
1/1  0s 34ms/step
1/1  0s 31ms/step
1/1  0s 34ms/step
1/1  0s 34ms/step
1/1  0s 39ms/step
1/1  0s 34ms/step
1/1  0s 37ms/step

1/1	—————	0s	34ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	38ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	37ms/step
1/1	—————	0s	31ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	42ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	41ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	37ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	31ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	37ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	43ms/step
1/1	—————	0s	38ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	42ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	41ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	40ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	37ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	35ms/step

1/1	—————	0s	33ms/step
1/1	—————	0s	37ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	37ms/step
1/1	—————	0s	37ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	31ms/step
1/1	—————	0s	46ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	37ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	37ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	39ms/step
1/1	—————	0s	38ms/step
1/1	—————	0s	47ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	38ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	37ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	38ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	63ms/step
1/1	—————	0s	65ms/step
1/1	—————	0s	46ms/step
1/1	—————	0s	49ms/step
1/1	—————	0s	53ms/step
1/1	—————	0s	54ms/step
1/1	—————	0s	45ms/step
1/1	—————	0s	39ms/step
1/1	—————	0s	60ms/step
1/1	—————	0s	52ms/step
1/1	—————	0s	42ms/step
1/1	—————	0s	52ms/step
1/1	—————	0s	45ms/step
1/1	—————	0s	40ms/step
1/1	—————	0s	61ms/step
1/1	—————	0s	44ms/step
1/1	—————	0s	41ms/step

1/1	—————	0s	54ms/step
1/1	—————	0s	46ms/step
1/1	—————	0s	41ms/step
1/1	—————	0s	57ms/step
1/1	—————	0s	54ms/step
1/1	—————	0s	59ms/step
1/1	—————	0s	54ms/step
1/1	—————	0s	55ms/step
1/1	—————	0s	47ms/step
1/1	—————	0s	50ms/step
1/1	—————	0s	52ms/step
1/1	—————	0s	49ms/step
1/1	—————	0s	56ms/step
1/1	—————	0s	54ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	37ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	40ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	38ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	37ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	40ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	39ms/step
1/1	—————	0s	32ms/step
1/1	—————	0s	38ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	39ms/step
1/1	—————	0s	31ms/step
1/1	—————	0s	39ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	38ms/step

1/1		0s	34ms/step
1/1		0s	37ms/step
1/1		0s	37ms/step
1/1		0s	34ms/step
1/1		0s	33ms/step
1/1		0s	35ms/step
1/1		0s	33ms/step
1/1		0s	35ms/step
1/1		0s	46ms/step
1/1		0s	36ms/step
1/1		0s	35ms/step
1/1		0s	37ms/step
1/1		0s	34ms/step
1/1		0s	35ms/step
1/1		0s	34ms/step
1/1		0s	35ms/step
1/1		0s	34ms/step
1/1		0s	34ms/step
1/1		0s	34ms/step
1/1		0s	32ms/step
1/1		0s	37ms/step
1/1		0s	34ms/step
1/1		0s	37ms/step
1/1		0s	35ms/step
1/1		0s	37ms/step
1/1		0s	34ms/step
1/1		0s	36ms/step
1/1		0s	35ms/step
1/1		0s	35ms/step
1/1		0s	34ms/step
1/1		0s	36ms/step
1/1		0s	32ms/step
1/1		0s	36ms/step
1/1		0s	34ms/step
1/1		0s	39ms/step
1/1		0s	36ms/step
1/1		0s	36ms/step
1/1		0s	38ms/step
1/1		0s	35ms/step
1/1		0s	35ms/step
1/1		0s	37ms/step
1/1		0s	34ms/step
1/1		0s	35ms/step
1/1		0s	34ms/step
1/1		0s	40ms/step
1/1		0s	36ms/step
1/1		0s	36ms/step
1/1		0s	35ms/step
1/1		0s	37ms/step
1/1		0s	35ms/step
1/1		0s	38ms/step
1/1		0s	34ms/step
1/1		0s	35ms/step
1/1		0s	33ms/step
1/1		0s	37ms/step

1/1	=====	0s	35ms/step
1/1	=====	0s	38ms/step
1/1	=====	0s	36ms/step
1/1	=====	0s	39ms/step
1/1	=====	0s	34ms/step
1/1	=====	0s	37ms/step
1/1	=====	0s	40ms/step
1/1	=====	0s	36ms/step
1/1	=====	0s	33ms/step
1/1	=====	0s	41ms/step
1/1	=====	0s	35ms/step
1/1	=====	0s	41ms/step
1/1	=====	0s	33ms/step
1/1	=====	0s	37ms/step
1/1	=====	0s	42ms/step
1/1	=====	0s	38ms/step
1/1	=====	0s	36ms/step
1/1	=====	0s	39ms/step
1/1	=====	0s	35ms/step
1/1	=====	0s	36ms/step
1/1	=====	0s	33ms/step
1/1	=====	0s	37ms/step
1/1	=====	0s	34ms/step
1/1	=====	0s	37ms/step
1/1	=====	0s	33ms/step
1/1	=====	0s	40ms/step
1/1	=====	0s	36ms/step
1/1	=====	0s	54ms/step
1/1	=====	0s	56ms/step
1/1	=====	0s	55ms/step
1/1	=====	0s	58ms/step

Accuracy 100.00

Checking 25 way accuracy....

1/1	=====	0s	45ms/step
1/1	=====	0s	44ms/step
1/1	=====	0s	56ms/step
1/1	=====	0s	56ms/step
1/1	=====	0s	56ms/step
1/1	=====	0s	50ms/step
1/1	=====	0s	51ms/step
1/1	=====	0s	52ms/step
1/1	=====	0s	54ms/step
1/1	=====	0s	57ms/step
1/1	=====	0s	58ms/step
1/1	=====	0s	45ms/step
1/1	=====	0s	54ms/step
1/1	=====	0s	60ms/step
1/1	=====	0s	50ms/step
1/1	=====	0s	56ms/step
1/1	=====	0s	59ms/step
1/1	=====	0s	51ms/step
1/1	=====	0s	55ms/step
1/1	=====	0s	57ms/step
1/1	=====	0s	58ms/step
1/1	=====	0s	44ms/step
1/1	=====	0s	49ms/step

1/1	_____	0s	46ms/step
1/1	_____	0s	50ms/step
1/1	_____	0s	58ms/step
1/1	_____	0s	66ms/step
1/1	_____	0s	35ms/step
1/1	_____	0s	34ms/step
1/1	_____	0s	32ms/step
1/1	_____	0s	38ms/step
1/1	_____	0s	35ms/step
1/1	_____	0s	38ms/step
1/1	_____	0s	34ms/step
1/1	_____	0s	38ms/step
1/1	_____	0s	36ms/step
1/1	_____	0s	36ms/step
1/1	_____	0s	34ms/step
1/1	_____	0s	35ms/step
1/1	_____	0s	34ms/step
1/1	_____	0s	37ms/step
1/1	_____	0s	33ms/step
1/1	_____	0s	37ms/step
1/1	_____	0s	34ms/step
1/1	_____	0s	37ms/step
1/1	_____	0s	34ms/step
1/1	_____	0s	37ms/step
1/1	_____	0s	35ms/step
1/1	_____	0s	37ms/step
1/1	_____	0s	35ms/step
1/1	_____	0s	35ms/step
1/1	_____	0s	36ms/step
1/1	_____	0s	38ms/step
1/1	_____	0s	42ms/step
1/1	_____	0s	41ms/step
1/1	_____	0s	43ms/step
1/1	_____	0s	37ms/step
1/1	_____	0s	35ms/step
1/1	_____	0s	40ms/step
1/1	_____	0s	35ms/step
1/1	_____	0s	36ms/step
1/1	_____	0s	34ms/step
1/1	_____	0s	39ms/step
1/1	_____	0s	38ms/step
1/1	_____	0s	34ms/step
1/1	_____	0s	34ms/step
1/1	_____	0s	38ms/step
1/1	_____	0s	37ms/step
1/1	_____	0s	41ms/step
1/1	_____	0s	35ms/step
1/1	_____	0s	39ms/step
1/1	_____	0s	34ms/step
1/1	_____	0s	37ms/step
1/1	_____	0s	34ms/step
1/1	_____	0s	37ms/step
1/1	_____	0s	33ms/step
1/1	_____	0s	42ms/step
1/1	_____	0s	31ms/step
1/1	_____	0s	39ms/step

1/1		0s	36ms/step
1/1		0s	38ms/step
1/1		0s	35ms/step
1/1		0s	38ms/step
1/1		0s	37ms/step
1/1		0s	39ms/step
1/1		0s	34ms/step
1/1		0s	36ms/step
1/1		0s	34ms/step
1/1		0s	49ms/step
1/1		0s	32ms/step
1/1		0s	38ms/step
1/1		0s	36ms/step
1/1		0s	42ms/step
1/1		0s	34ms/step
1/1		0s	38ms/step
1/1		0s	36ms/step
1/1		0s	39ms/step
1/1		0s	34ms/step
1/1		0s	35ms/step
1/1		0s	32ms/step
1/1		0s	48ms/step
1/1		0s	33ms/step
1/1		0s	38ms/step
1/1		0s	35ms/step
1/1		0s	38ms/step
1/1		0s	36ms/step
1/1		0s	39ms/step
1/1		0s	36ms/step
1/1		0s	36ms/step
1/1		0s	37ms/step
1/1		0s	37ms/step
1/1		0s	37ms/step
1/1		0s	39ms/step
1/1		0s	40ms/step
1/1		0s	38ms/step
1/1		0s	36ms/step
1/1		0s	40ms/step
1/1		0s	38ms/step
1/1		0s	40ms/step
1/1		0s	38ms/step
1/1		0s	38ms/step
1/1		0s	33ms/step
1/1		0s	39ms/step
1/1		0s	33ms/step
1/1		0s	40ms/step
1/1		0s	36ms/step
1/1		0s	39ms/step
1/1		0s	36ms/step
1/1		0s	39ms/step
1/1		0s	37ms/step
1/1		0s	40ms/step
1/1		0s	34ms/step
1/1		0s	36ms/step
1/1		0s	34ms/step
1/1		0s	35ms/step

1/1	—————	0s	35ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	40ms/step
1/1	—————	0s	37ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	38ms/step
1/1	—————	0s	38ms/step
1/1	—————	0s	37ms/step
1/1	—————	0s	34ms/step
1/1	—————	0s	37ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	46ms/step
1/1	—————	0s	52ms/step
1/1	—————	0s	43ms/step
1/1	—————	0s	55ms/step
1/1	—————	0s	49ms/step
1/1	—————	0s	40ms/step
1/1	—————	0s	56ms/step
1/1	—————	0s	42ms/step
1/1	—————	0s	50ms/step
1/1	—————	0s	40ms/step
1/1	—————	0s	45ms/step
1/1	—————	0s	57ms/step
1/1	—————	0s	62ms/step
1/1	—————	0s	44ms/step
1/1	—————	0s	53ms/step
1/1	—————	0s	51ms/step
1/1	—————	0s	43ms/step
1/1	—————	0s	43ms/step
1/1	—————	0s	54ms/step
1/1	—————	0s	57ms/step
1/1	—————	0s	51ms/step
1/1	—————	0s	52ms/step
1/1	—————	0s	49ms/step
1/1	—————	0s	45ms/step
1/1	—————	0s	45ms/step
1/1	—————	0s	45ms/step
1/1	—————	0s	57ms/step
1/1	—————	0s	50ms/step
1/1	—————	0s	64ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	38ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	39ms/step
1/1	—————	0s	35ms/step
1/1	—————	0s	38ms/step
1/1	—————	0s	36ms/step
1/1	—————	0s	37ms/step
1/1	—————	0s	41ms/step
1/1	—————	0s	37ms/step
1/1	—————	0s	33ms/step
1/1	—————	0s	41ms/step
1/1	—————	0s	38ms/step
1/1	—————	0s	37ms/step
1/1	—————	0s	37ms/step
1/1	—————	0s	39ms/step

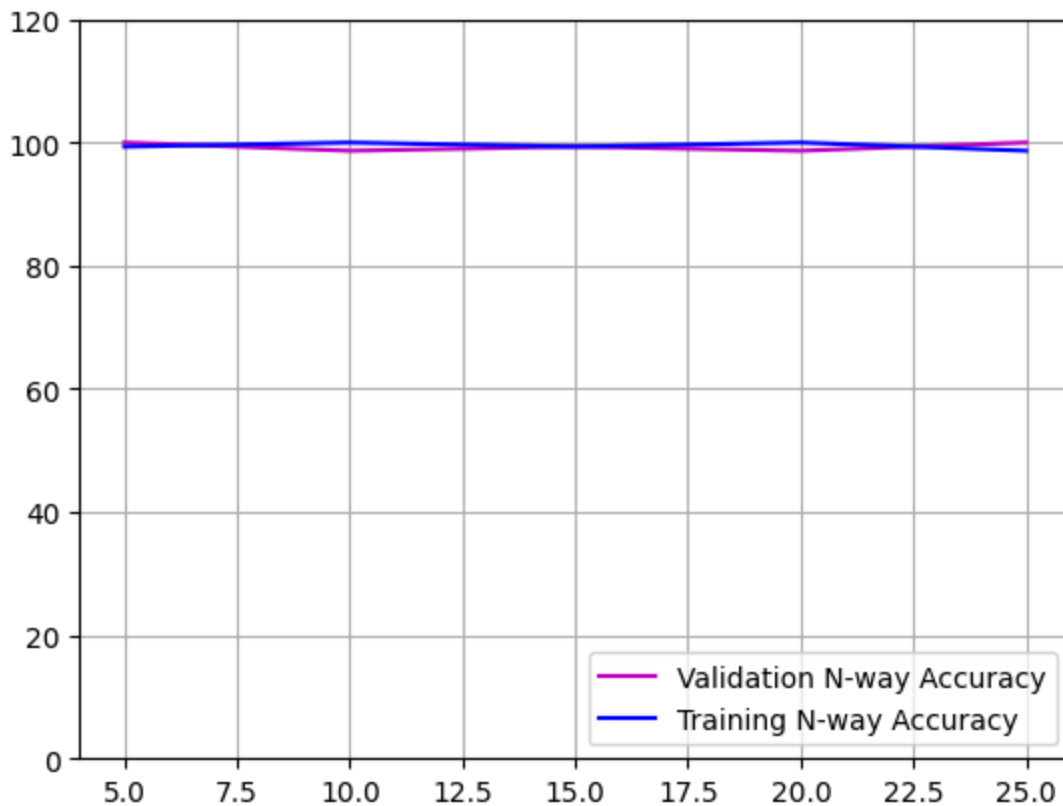
1/1		0s	34ms/step
1/1		0s	37ms/step
1/1		0s	34ms/step
1/1		0s	37ms/step
1/1		0s	35ms/step
1/1		0s	36ms/step
1/1		0s	34ms/step
1/1		0s	39ms/step
1/1		0s	35ms/step
1/1		0s	41ms/step
1/1		0s	36ms/step
1/1		0s	38ms/step
1/1		0s	34ms/step
1/1		0s	41ms/step
1/1		0s	35ms/step
1/1		0s	37ms/step
1/1		0s	34ms/step
1/1		0s	39ms/step
1/1		0s	34ms/step
1/1		0s	38ms/step
1/1		0s	35ms/step
1/1		0s	38ms/step
1/1		0s	37ms/step
1/1		0s	40ms/step
1/1		0s	35ms/step
1/1		0s	39ms/step
1/1		0s	34ms/step
1/1		0s	38ms/step
1/1		0s	34ms/step
1/1		0s	35ms/step
1/1		0s	35ms/step
1/1		0s	36ms/step
1/1		0s	35ms/step
1/1		0s	38ms/step
1/1		0s	45ms/step
1/1		0s	38ms/step
1/1		0s	36ms/step
1/1		0s	37ms/step
1/1		0s	33ms/step
1/1		0s	39ms/step
1/1		0s	34ms/step
1/1		0s	35ms/step
1/1		0s	32ms/step
1/1		0s	39ms/step
1/1		0s	37ms/step
1/1		0s	38ms/step
1/1		0s	35ms/step
1/1		0s	38ms/step
1/1		0s	33ms/step
1/1		0s	37ms/step
1/1		0s	33ms/step
1/1		0s	37ms/step
1/1		0s	36ms/step
1/1		0s	37ms/step
1/1		0s	36ms/step
1/1		0s	37ms/step

1/1	0s	35ms/step
1/1	0s	39ms/step
1/1	0s	35ms/step
1/1	0s	40ms/step
1/1	0s	37ms/step
1/1	0s	39ms/step
1/1	0s	34ms/step
1/1	0s	38ms/step
1/1	0s	36ms/step
1/1	0s	39ms/step
1/1	0s	32ms/step
1/1	0s	38ms/step
1/1	0s	35ms/step
1/1	0s	39ms/step
1/1	0s	35ms/step
1/1	0s	38ms/step
1/1	0s	40ms/step
1/1	0s	37ms/step
1/1	0s	35ms/step
1/1	0s	36ms/step
1/1	0s	34ms/step
1/1	0s	40ms/step
1/1	0s	33ms/step
1/1	0s	39ms/step
1/1	0s	37ms/step
1/1	0s	38ms/step
1/1	0s	37ms/step
1/1	0s	49ms/step
1/1	0s	34ms/step
1/1	0s	36ms/step
1/1	0s	34ms/step
1/1	0s	38ms/step
1/1	0s	37ms/step
1/1	0s	36ms/step
1/1	0s	34ms/step
1/1	0s	37ms/step
1/1	0s	34ms/step
1/1	0s	38ms/step
1/1	0s	36ms/step
1/1	0s	37ms/step
1/1	0s	35ms/step
1/1	0s	43ms/step
1/1	0s	33ms/step
1/1	0s	37ms/step
1/1	0s	36ms/step
1/1	0s	36ms/step
1/1	0s	57ms/step
1/1	0s	48ms/step
1/1	0s	46ms/step
1/1	0s	54ms/step
1/1	0s	81ms/step
1/1	0s	55ms/step
1/1	0s	46ms/step

Accuracy 98.67

8.3 N-way Accuracy Plot

```
In [24]: # Test & Train set accuracy Plot
plt.plot(nway_list, test_acc, 'm', label='Validation N-way Accuracy')
plt.plot(nway_list, train_acc, 'b', label='Training N-way Accuracy')
plt.axis([None, None, 0, 120])
plt.grid(True)
plt.legend(loc='lower right')
# It was written as plt.show, this shows the documentation, I fixed it by changing
plt.show()
```



Share your observation from the graph:

From the plot, both the Training and Validation N-way accuracy curves stay very close to each other and remain very high across all N-way levels (5, 10, 15, 20, 25). Which means:

- Accuracy is consistently high (around ~ 100%) for both training and validation, even when the N-way task becomes harder. This means the model is very reliable and consistent most of the times.
- Training and validation curves almost overlap. This shows that the model is not overfitting and the behavior on unseen data is almost identical to training data.
- Slight fluctuations at different N-way values between train and validation sets which is perfectly normal.
- Model generalized well as it performs almost identical on unseen validation dataj with a very minor fluctuations.

In short, the N-way accuracy plot shows that the Siamese network is performing extremely well. It maintains almost perfect accuracy even as the number of comparisons increases, and it generalizes to new images without overfitting.

8. Conclusions and Insights (10 Marks)

What are your conclusions and insights from this exercise? What recommendations would you take to the business if they were interested in converting this into a full-fledged solution?

Write your answer here:

This exercise showed how a small Siamese Network can be used to recognize whether two face images belong to the same person by learning good-quality embeddings. Even though the model was trained on a relatively small dataset, it still learned to separate faces very well using Triplet Loss. The training loss went down smoothly, and both the training and validation N-way accuracies stayed extremely high, which means the model generalized well to unseen images.

The N-way accuracy results also showed that the model remained accurate even when the number of comparison images increased. This tells us that the embeddings produced by the model are strong and consistent. The network learned the patterns of faces in a way that helps it correctly match or reject images even as the task becomes more difficult.

Overall, this exercise demonstrates that a properly trained Siamese Network is a very effective approach for face-similarity or identity-matching tasks.

Business Recommendations:

If a company wanted to turn this into a full-fledged solution, the following steps would be important:

1. To handle real-world variations such as lighting, pose, age changes, and background differences, the model should be trained on a much bigger and more diverse face dataset. More data leads to more reliable embeddings.
2. The current method of triplet selection is to pick triplets randomly, which is sufficient for demonstration but in real system, it needs to be trained on more harder to differentiate examples. This will improve the accuracy to differentiate between more closely related yet different images.
3. The real life images might not be available in high definition and in proper frame alignment, angles, lighting, and isolation. Hence, proper handling for such scenarios (Pre-Processing) are important.

4. For a commercial quality model, we might use an already pre-trained models like FaceNet or MobileFaceNet. As they are already trained on large datasets and are widely used in industry.
5. Since face recognition involves sensitive personal data, the system must utilize privacy standards and policies such as Encryption, Access Control, Secure Storage, Proper Auditing, etc.